Tugas Pertemuan 5 – Introduction to Data Definition Language

Mata Kuliah: Sistem Basis Data (2026)

Prodi: Sarjana Terapan-Teknik Informatika POLBAN

Kelas: 2B

NIM: 251524048

Nama Lengkap: Ghaisan Khoirul Badruzaman

Prasyarat

1. Menggunakan alat bantu atau perangkat lunak RDBMS PostgreSQL
2. Menggunakan GUI Client Tools seperti pgAdmin atau DBeaver (opsional)

Instruksi

1. Pelajari dan kerjakan syntax yang disediakan pada pgtutorial.com – Section 1 dan 3.
2. **Wajib dikerjakan secara terturut**!
3. Sertakan syntax yang dipraktikkan dalam bentuk **teks** syntax SQL!
4. Sertakan **screenshot halaman penuh yang menyertakan taskbar dengan waktu** (Print Screen) untuk menunjukkan syntax dan hasil eksekusi query pada RDBMS!
5. Ekspor file lembar jawaban ini dalam format PDF dengan nama file:
   P5_<2X - Kelas>_<3 Digit Terakhir NIM>.pdf
   contoh sebagai berikut:
   P5_2Z_777_DjokoCahyoUtomiLieharyani.pdf
6. Kumpulkan melalui Ms. Teams sesuai dengan tenggat waktu yang telah ditentukan!

Jika ada hal yang ingin ditanyakan, silahkan sampaikan melalui Ms Teams.

Selamat mengerjakan!

Latihan No. 1

Judul
Section 1: What is PostgreSQL (masuk ke PostgreSQL dan membuat database)
Syntax

```
orb -m ciwaruga-uji
sudo -u postgres psql

\pset pager off

\pset null NULL

SELECT version();

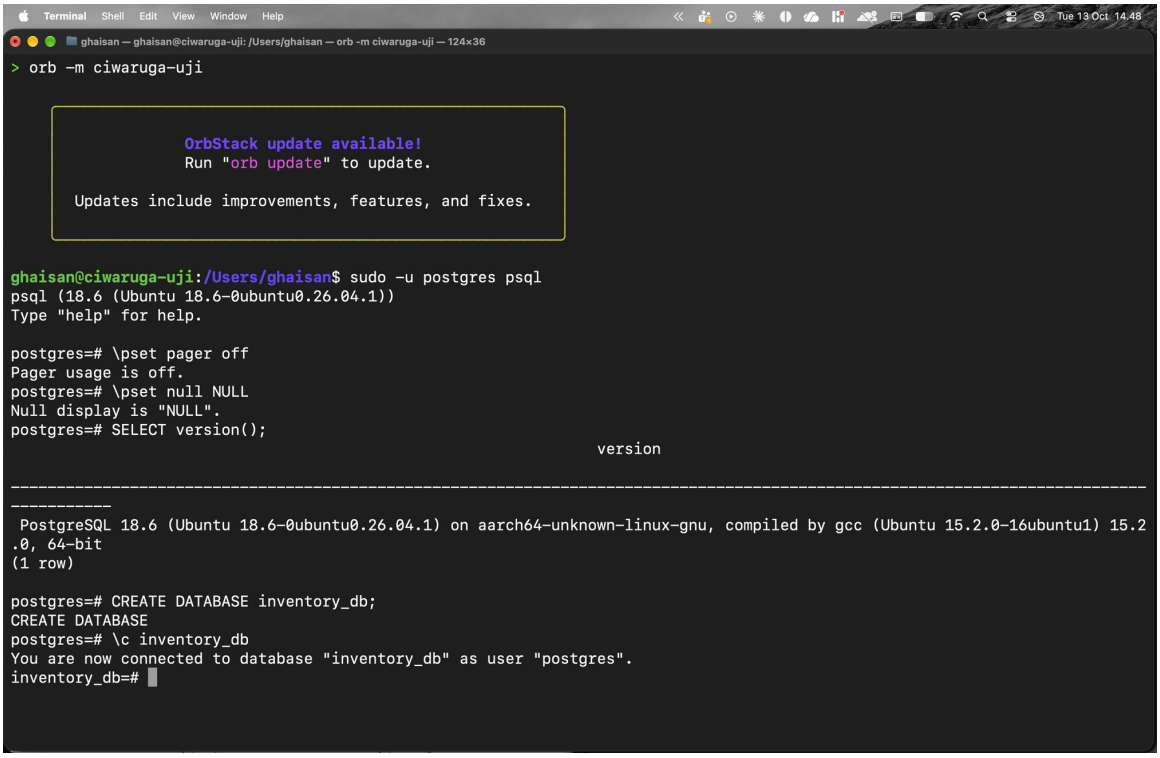
CREATE DATABASE inventory_db;

\c inventory_db
```

Keterangan

Materi What is PostgreSQL berisi pengenalan dan tidak memuat syntax. Di materi disebutkan bahwa untuk memakai PostgreSQL dibutuhkan client program, salah satunya program CLI. Di sini saya memakai psql, client CLI bawaan PostgreSQL, di mesin Ubuntu (OrbStack) yang sudah terpasang PostgreSQL 18.6.
orb -m ciwaruga-uji masuk ke mesin Ubuntu, lalu sudo -u postgres psql masuk ke PostgreSQL sebagai user postgres. \pset pager off membuat hasil query langsung tampil di layar (tidak dibuka di pager), dan \pset null NULL membuat nilai NULL tampil sebagai tulisan NULL seperti pada materi. SELECT version() menampilkan versi server, yaitu PostgreSQL 18.6. Terakhir dibuat database inventory_db untuk semua latihan berikutnya, lalu pindah ke database tersebut dengan \c inventory_db.

Hasil Eksekusi (Screenshot)



Latihan No. 2

Judul
Section 1: Creating a new table
Syntax

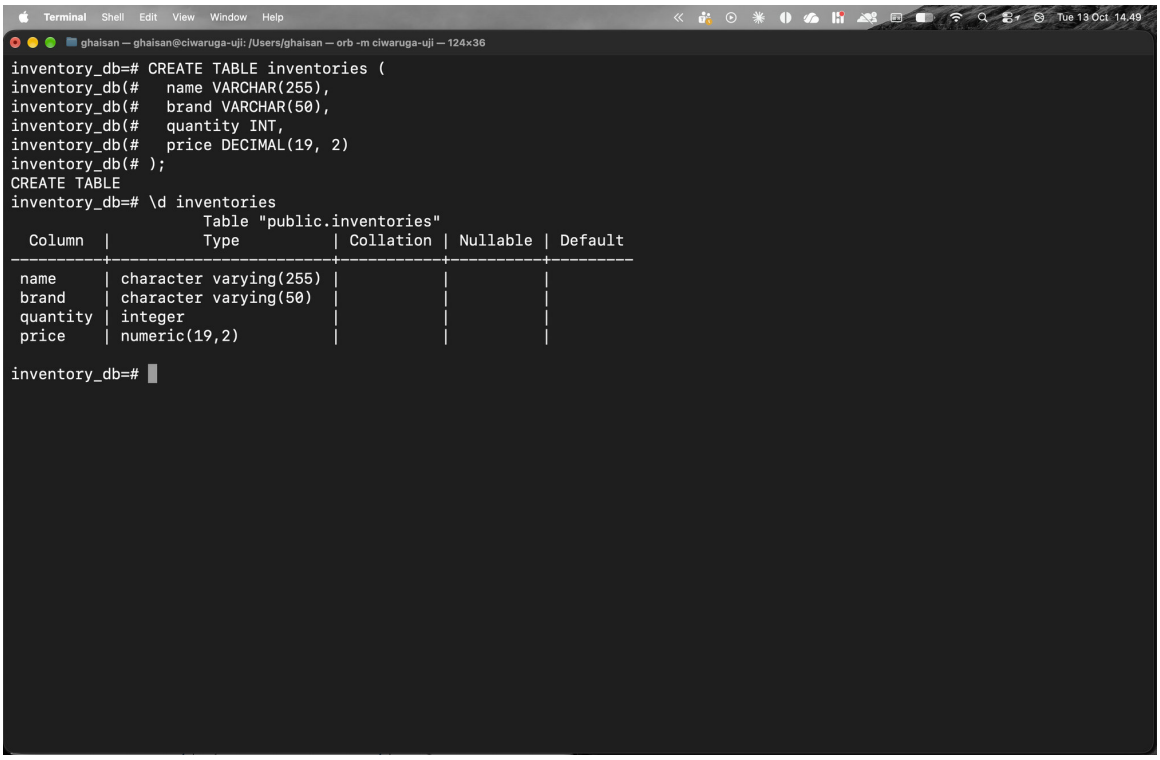
```
CREATE TABLE inventories (
  name VARCHAR(255),
  brand VARCHAR(50),
  quantity INT,
  price DECIMAL(19, 2)
);

\d inventories
```

Keterangan

CREATE TABLE membuat tabel inventories dengan empat kolom sesuai rancangan di materi: name dan brand bertipe VARCHAR (string dengan panjang maksimal 255 dan 50 karakter), quantity bertipe INT, dan price bertipe DECIMAL(19, 2), yaitu angka desimal sampai 19 digit dengan 2 digit di belakang koma. Nama tabel memakai snake case dan bentuk jamak sesuai konvensi.
PostgreSQL membalas CREATE TABLE yang artinya tabel berhasil dibuat. \d inventories dipakai untuk melihat struktur tabel. Terlihat VARCHAR disimpan sebagai character varying, INT sebagai integer, dan DECIMAL sebagai numeric(19,2), dan semua kolom masih boleh NULL karena belum diberi constraint.

Hasil Eksekusi (Screenshot)



Latihan No. 3

Judul
Section 1: Inserting data into a table

Syntax

```
CREATE TABLE IF NOT EXISTS inventories (
  name VARCHAR(255),
  brand VARCHAR(50),
  quantity INT,
  price DECIMAL(19, 2)
);

INSERT INTO
  inventories (name, brand, quantity, price)
VALUES
  ('iPhone 14 Pro', 'Apple', 10, 999.99);

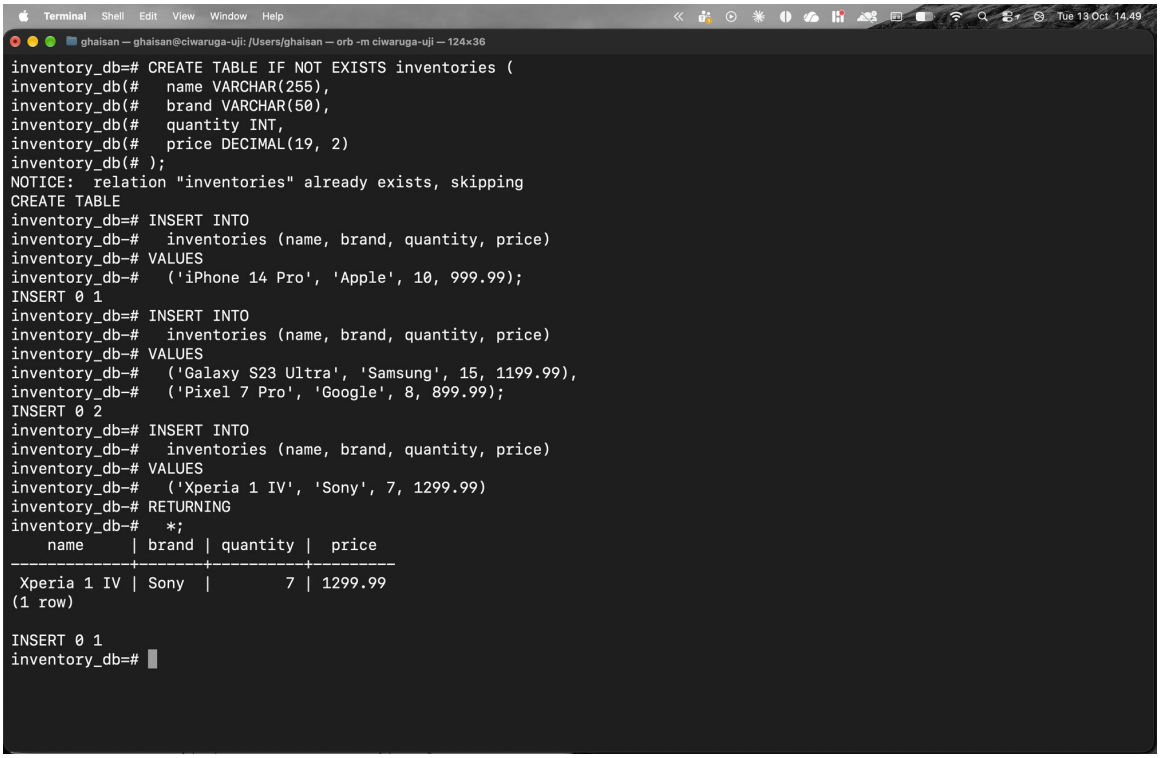
INSERT INTO
  inventories (name, brand, quantity, price)
VALUES
  ('Galaxy S23 Ultra', 'Samsung', 15, 1199.99),
  ('Pixel 7 Pro', 'Google', 8, 899.99);

INSERT INTO
  inventories (name, brand, quantity, price)
VALUES
  ('Xperia 1 IV', 'Sony', 7, 1299.99)
RETURNING
  *;
```

Keterangan

CREATE TABLE IF NOT EXISTS dijalankan lagi sesuai materi. Karena tabel inventories sudah dibuat di latihan sebelumnya, PostgreSQL hanya memberi NOTICE relation "inventories" already exists, skipping dan tidak error.
INSERT pertama menambahkan satu baris (iPhone 14 Pro) dengan hasil INSERT 0 1. INSERT kedua menambahkan dua baris sekaligus dengan menuliskan beberapa tuple di VALUES, hasilnya INSERT 0 2. INSERT ketiga memakai RETURNING * sehingga baris yang baru masuk (Xperia 1 IV, Sony, 7, 1299.99) langsung ditampilkan. Angka terakhir pada INSERT 0 n adalah jumlah baris yang berhasil dimasukkan.

Hasil Eksekusi (Screenshot)



Latihan No. 4

Judul
Section 1: Selecting data from a table

Syntax

```
SELECT * FROM inventories;

SELECT
  name,
  price
FROM
  inventories;

SELECT
  name,
  quantity * price
FROM
  inventories;

SELECT
  name,
  quantity * price AS amount
FROM
  inventories;

SELECT
  name,
  quantity * price amount
FROM
  inventories;

SELECT
  name AS product_name,
```

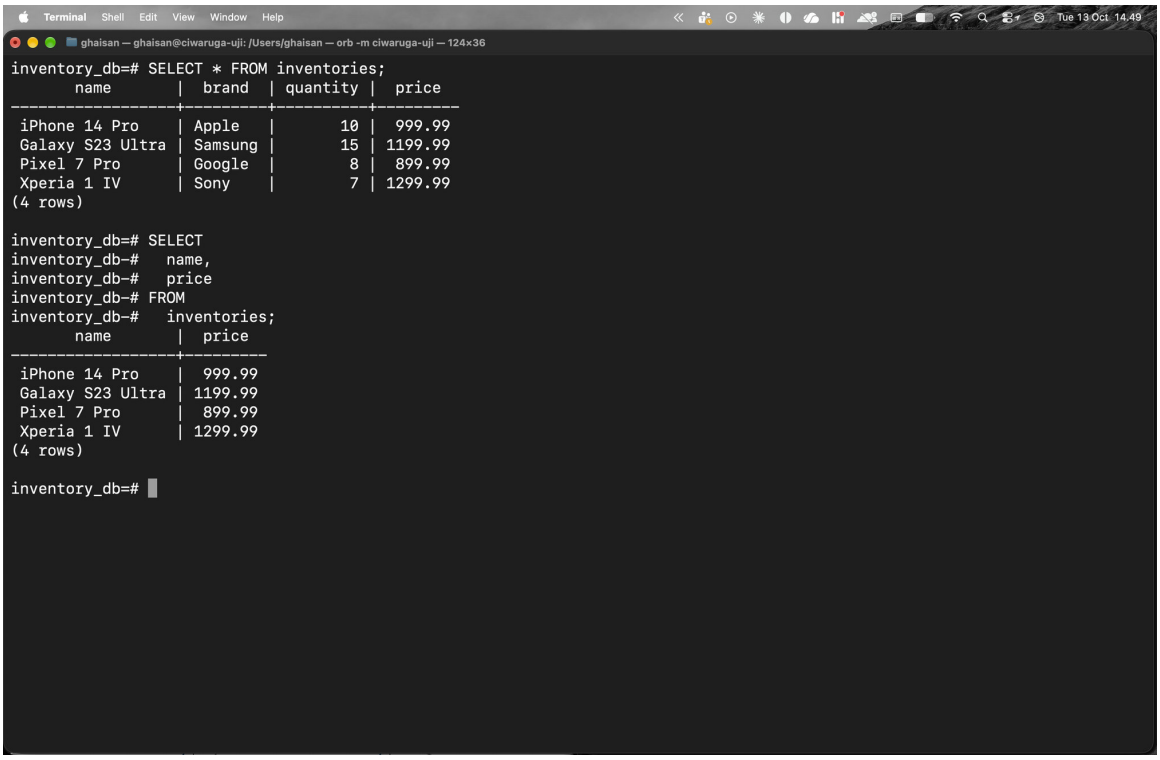
```
  quantity * price AS amount
FROM
  inventories;
```

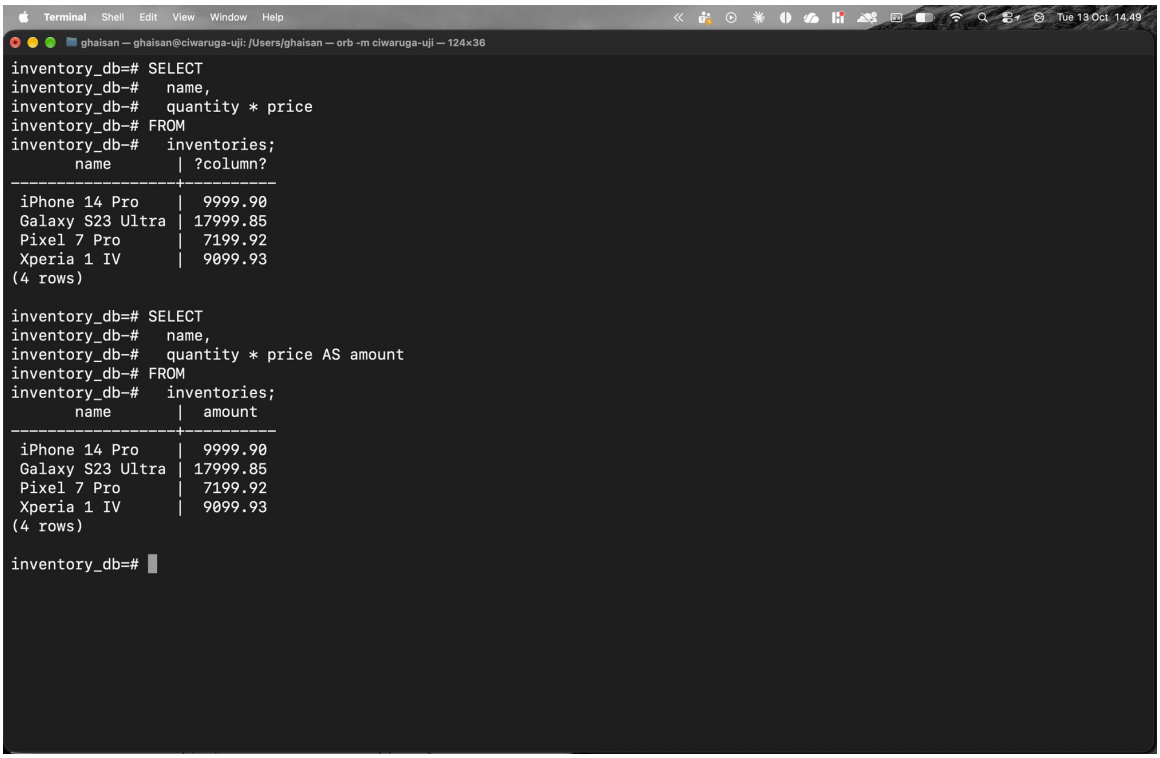
Keterangan

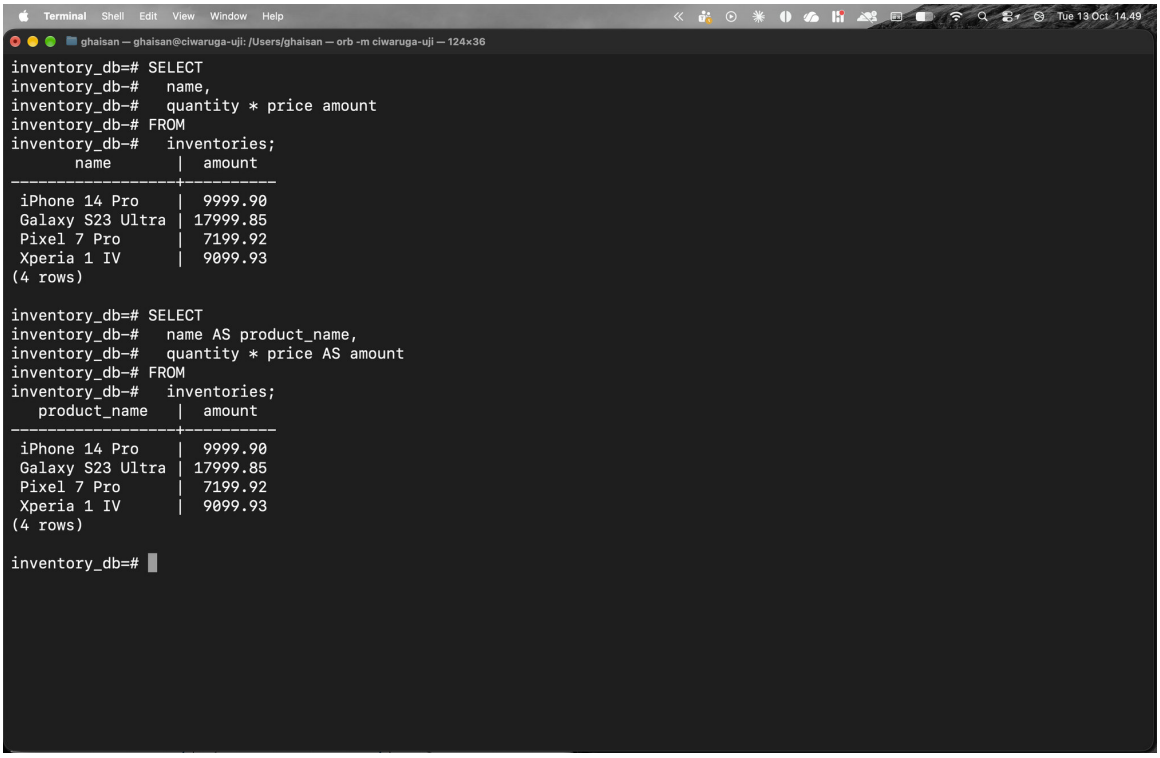
SELECT * mengambil semua kolom dari tabel inventories, hasilnya 4 baris yang dimasukkan pada latihan 3. SELECT name, price hanya mengambil dua kolom. SELECT name, quantity * price menghitung nilai stok tiap produk. Kolom hasil perhitungan belum punya nama, jadi PostgreSQL menamainya ?column?.

Kolom perhitungan itu lalu diberi alias amount memakai AS, dan dicoba juga tanpa AS (quantity * price amount) yang hasilnya sama. Query terakhir memberi alias product_name ke kolom name. Contoh hasilnya, iPhone 14 Pro punya quantity 10 dan price 999.99 sehingga amount = 9999.90.

Hasil Eksekusi (Screenshot)




Terminal Shell Edit View Window Help
Tue 13 Oct 14.49
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# SELECT
inventory_db-#   name,
inventory_db-#   quantity * price
inventory_db-# FROM
inventory_db-#   inventories;
name | ?column?
iPhone 14 Pro | 9999.90
Galaxy S23 Ultra | 17999.85
Pixel 7 Pro | 7199.92
Xperia 1 IV | 9099.93
(4 rows)
inventory_db=# SELECT
inventory_db-#   name,
inventory_db-#   quantity * price AS amount
inventory_db-# FROM
inventory_db-#   inventories;
name | amount
iPhone 14 Pro | 9999.90
Galaxy S23 Ultra | 17999.85
Pixel 7 Pro | 7199.92
Xperia 1 IV | 9099.93
(4 rows)
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.49
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# SELECT
inventory_db-#   name,
inventory_db-#   quantity * price amount
inventory_db-# FROM
inventory_db-#   inventories;
name | amount
iPhone 14 Pro | 9999.90
Galaxy S23 Ultra | 17999.85
Pixel 7 Pro | 7199.92
Xperia 1 IV | 9099.93
(4 rows)
inventory_db=# SELECT
inventory_db-#   name AS product_name,
inventory_db-#   quantity * price AS amount
inventory_db-# FROM
inventory_db-#   inventories;
product_name | amount
iPhone 14 Pro | 9999.90
Galaxy S23 Ultra | 17999.85
Pixel 7 Pro | 7199.92
Xperia 1 IV | 9099.93
(4 rows)
inventory_db=#

Latihan No. 5

Judul

Section 1: Concatenating strings

Syntax

```
SELECT
  brand || name AS product
FROM
  inventories;

SELECT
  brand || ' ' || name AS product
FROM
  inventories;

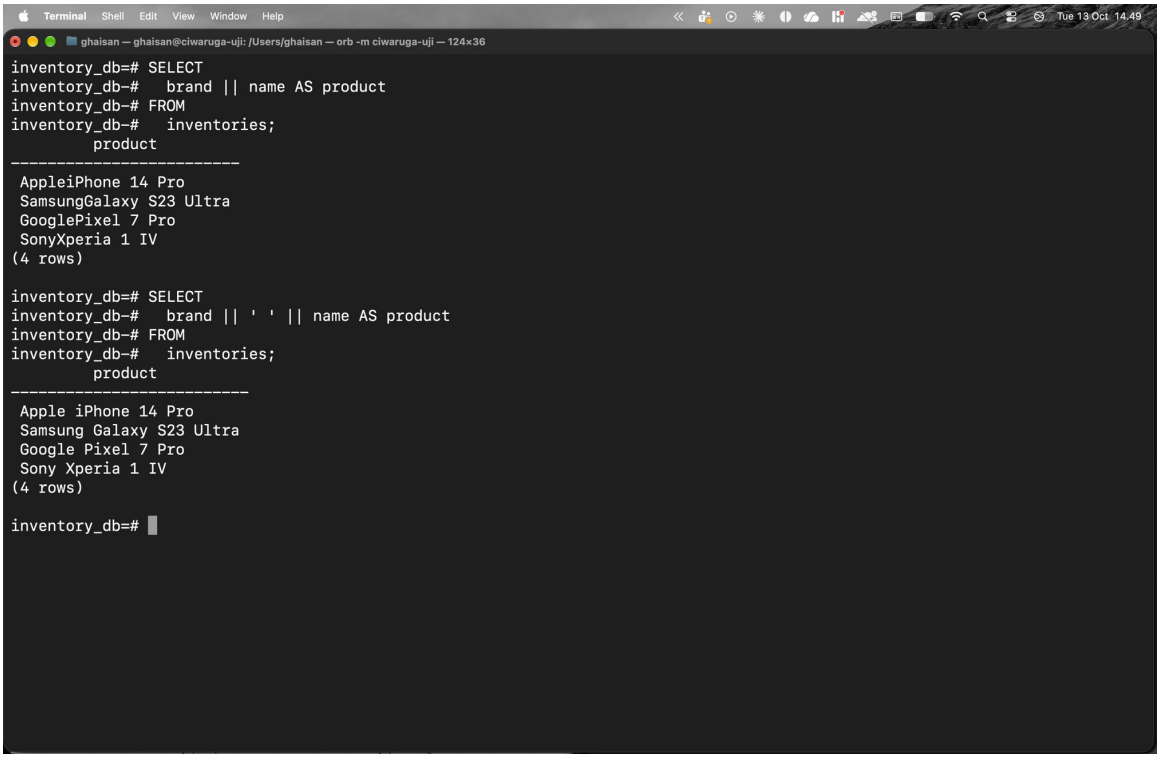
SELECT
  CONCAT(brand, ' ', name) AS product
FROM
  inventories;

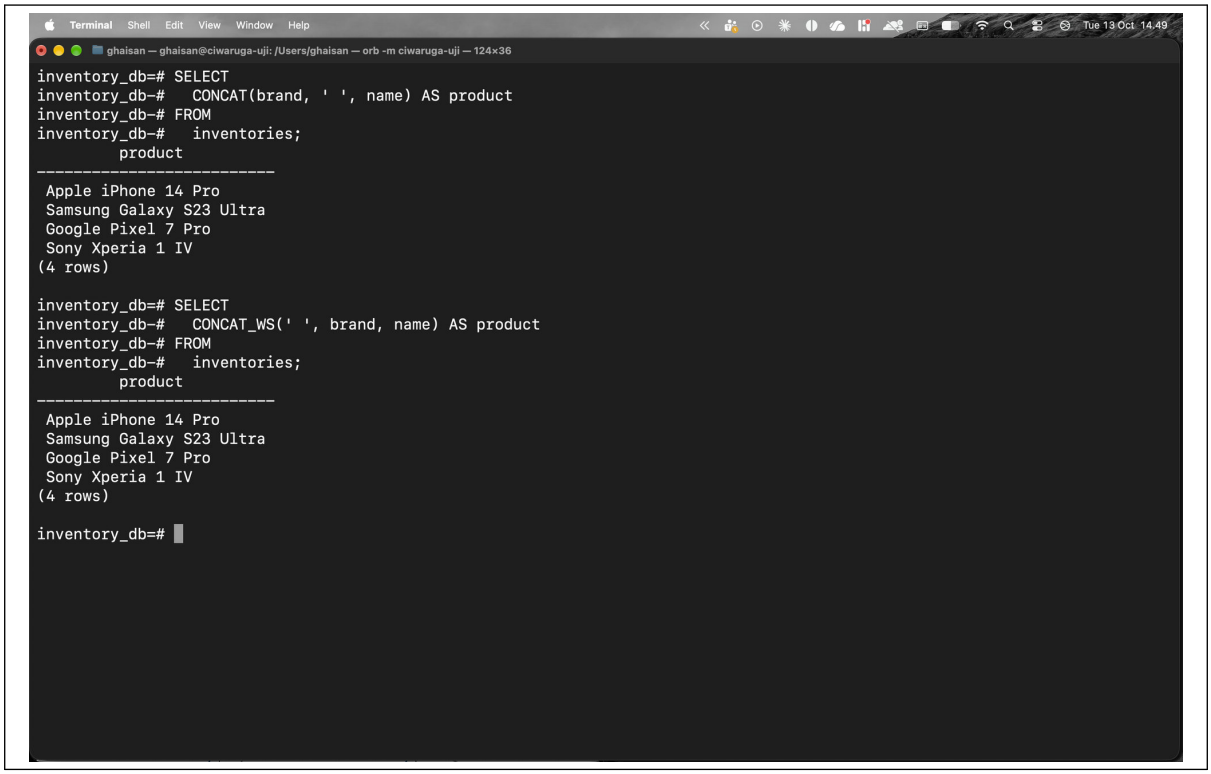
SELECT
  CONCAT_WS(' ', brand, name) AS product
FROM
  inventories;
```

Keterangan

Operator || menyambung dua string. brand || name menghasilkan AppleiPhone 14 Pro karena tidak ada pemisah, jadi pada query kedua ditambahkan ' ' (spasi) di tengahnya. Fungsi CONCAT(brand, ' ', name) memberi hasil yang sama. CONCAT_WS(' ', brand, name) juga sama, bedanya argumen pertama adalah pemisah (WS = with separator) yang otomatis dipasang di antara string berikutnya. Semua hasil diberi alias product.

Hasil Eksekusi (Screenshot)



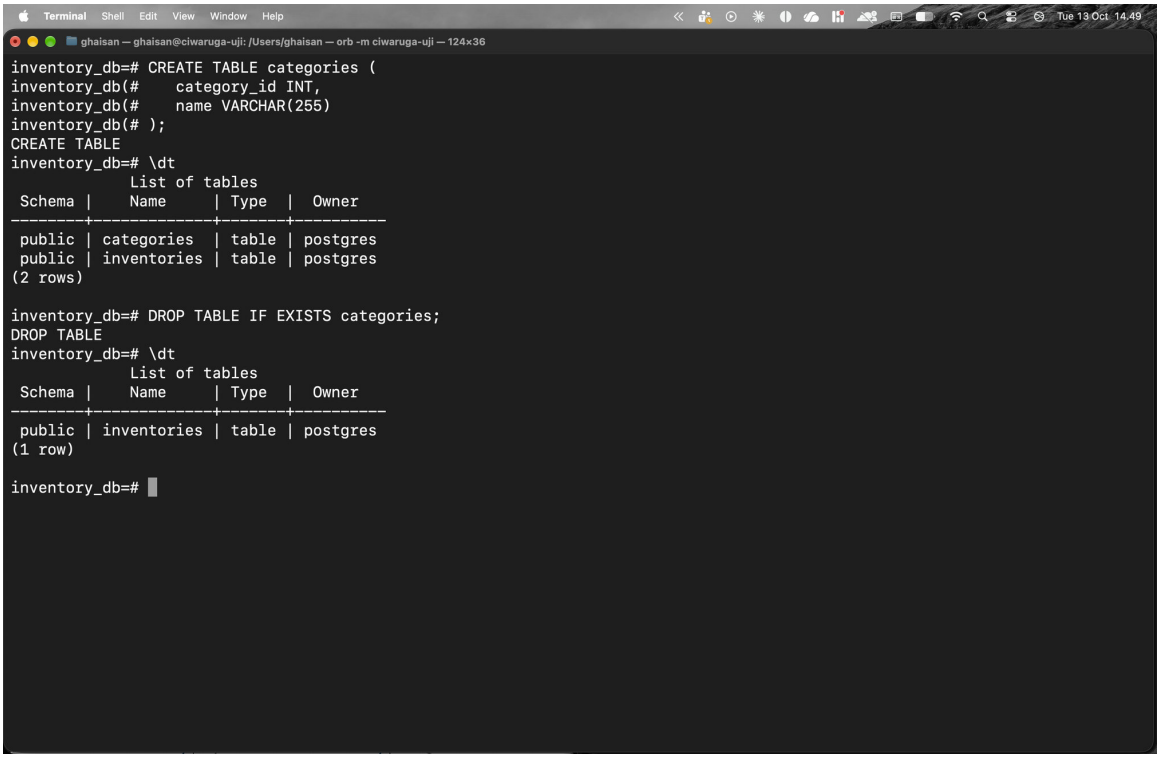


Latihan No. 6

Judul
Section 1: Dropping a table
Syntax

`CREATE TABLE categories (`
`  category_id INT,`
`  name VARCHAR(255)`
`);`
 	
`\dt`
 	
`DROP TABLE IF EXISTS categories;`
 	
`\dt`

Keterangan
Tabel categories dibuat lalu dicek dengan \dt, terlihat ada dua tabel (categories dan inventories). DROP TABLE IF EXISTS categories menghapus tabel categories beserta isinya. Setelah dicek lagi dengan \dt, tinggal tabel inventories. Opsi IF EXISTS membuat perintah tidak error kalau tabel yang mau dihapus ternyata tidak ada.

Hasil Eksekusi (Screenshot)


Latihan No. 7

Judul
Section 3: Database design
Syntax
Tidak ada syntax pada materi ini (materi berupa perancangan tabel dan relasi).
Keterangan
Materi ini berisi perancangan database untuk sistem inventory sederhana dan tidak memuat syntax SQL, jadi tidak ada query yang dijalankan. Tabel yang diidentifikasi adalah users, profiles, warehouses, tags, brands, categories, products, transactions, inventories, dan product_tags. Relasi users dengan profiles adalah one-to-one. Relasi one-to-many ada pada brands ke products, categories ke products, users ke transactions, warehouses ke transactions, dan products ke transactions, ditambah categories ke dirinya sendiri untuk subkategori. Relasi products dengan tags adalah many-to-many, dipecah menjadi dua relasi one-to-many lewat tabel penghubung product_tags. Contoh tabel pada latihan berikutnya (primary key, NOT NULL, DEFAULT, CHECK, UNIQUE, foreign key) diambil dari rancangan ini.
Hasil Eksekusi (Screenshot)
Tidak ada screenshot karena tidak ada query yang dieksekusi pada materi ini.

Latihan No. 8

Judul
Section 3: Primary key
Syntax

```
CREATE TABLE brands (
  brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(50)
);

INSERT INTO
  brands (name)
VALUES
  ('Apple')
RETURNING
  *;

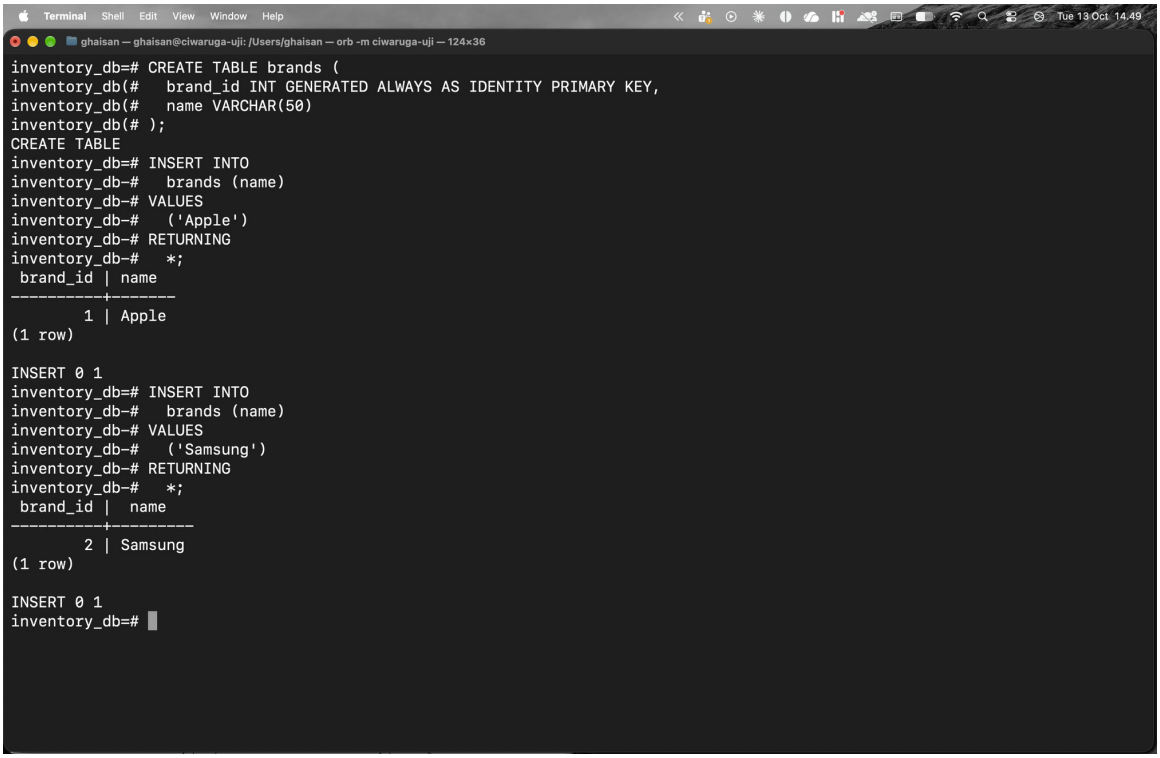
INSERT INTO
  brands (name)
VALUES
  ('Samsung')
RETURNING
  *;

CREATE TABLE product_tags (
  product_id INT,
  tag_id INT,
  PRIMARY KEY (product_id, tag_id)
);

\d product_tags
```

Keterangan
Tabel brands dibuat dengan brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, artinya brand_id menjadi primary key yang nilainya dibuat otomatis (auto increment). Karena itu INSERT cukup mengisi kolom name. Insert Apple menghasilkan brand_id 1 dan insert Samsung menghasilkan brand_id 2 (RETURNING * hanya menampilkan baris yang baru dimasukkan). Tabel product_tags memakai composite primary key, PRIMARY KEY (product_id, tag_id), yang ditulis sebagai table constraint. Dari \d product_tags terlihat kedua kolom otomatis menjadi not null dan ada index product_tags_pkey pada (product_id, tag_id).

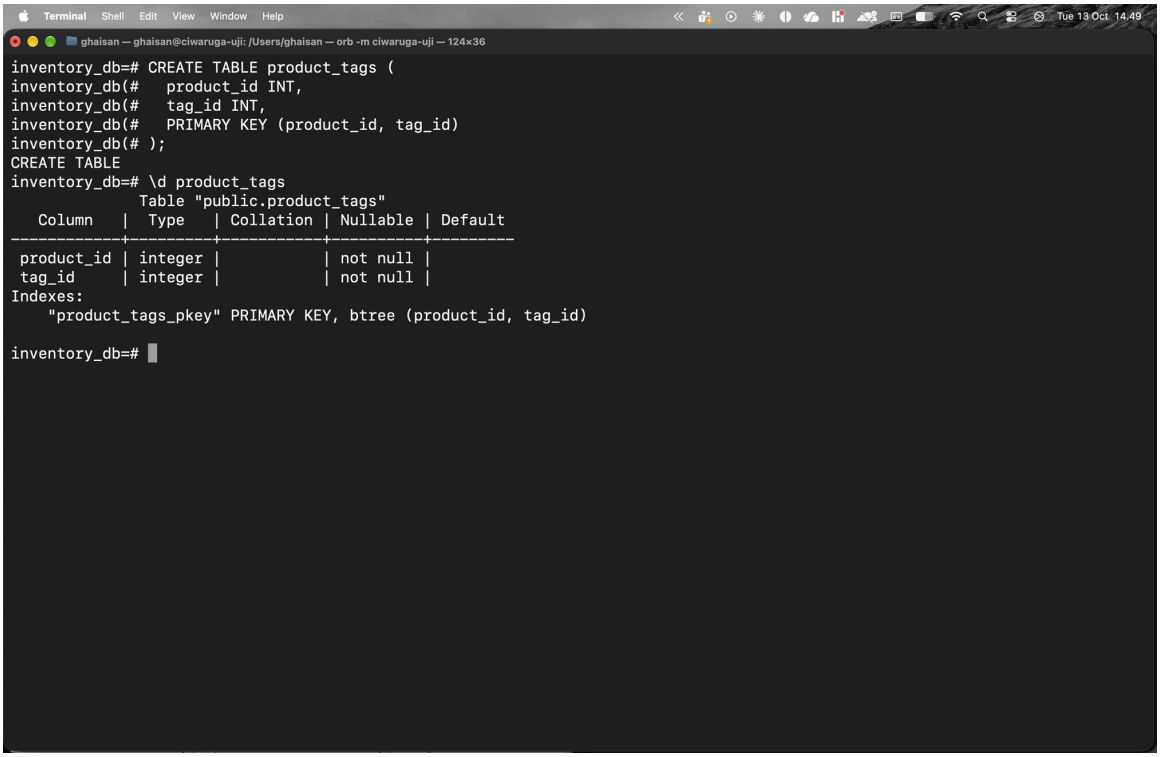
Hasil Eksekusi (Screenshot)



```
inventory_db=# CREATE TABLE brands (
inventory_db(#   brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(50)
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   brands (name)
inventory_db-# VALUES
inventory_db-#   ('Apple')
inventory_db-# RETURNING
inventory_db-#   *;
 brand_id | name
----------+-------
        1 | Apple
(1 row)

INSERT 0 1
inventory_db=# INSERT INTO
inventory_db-#   brands (name)
inventory_db-# VALUES
inventory_db-#   ('Samsung')
inventory_db-# RETURNING
inventory_db-#   *;
 brand_id |  name
----------+---------
        2 | Samsung
(1 row)

INSERT 0 1
inventory_db=#
```



```
inventory_db=# CREATE TABLE product_tags (
inventory_db(#   product_id INT,
inventory_db(#   tag_id INT,
inventory_db(#   PRIMARY KEY (product_id, tag_id)
inventory_db(# );
CREATE TABLE
inventory_db=# \d product_tags
             Table "public.product_tags"
   Column   |  Type   | Collation | Nullable | Default
------------+---------+-----------+----------+---------
 product_id | integer |           | not null |
 tag_id     | integer |           | not null |
Indexes:
    "product_tags_pkey" PRIMARY KEY, btree (product_id, tag_id)

inventory_db=#
```

Latihan No. 9

Judul
Section 3: NOT NULL constraint

Syntax

```
SELECT
  NULL = 1 AS result;

SELECT
  NULL = NULL AS result;

CREATE TABLE warehouses (
  warehouse_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(255),
  address VARCHAR(255)
);

INSERT INTO
  warehouses (name, address)
VALUES
  ('San Jose', NULL)
RETURNING
  name,
  address;

INSERT INTO
  warehouses (name)
VALUES
  ('San Francisco')
RETURNING
  name,
  address;

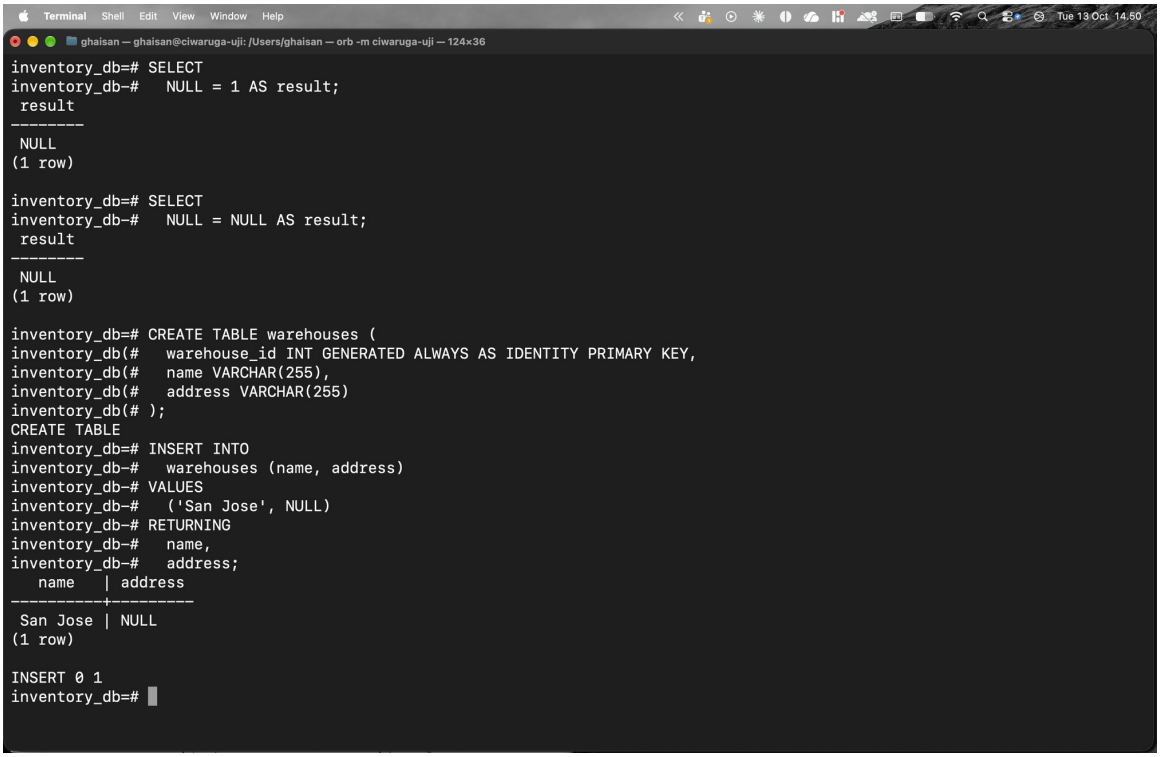
DROP TABLE IF EXISTS warehouses;

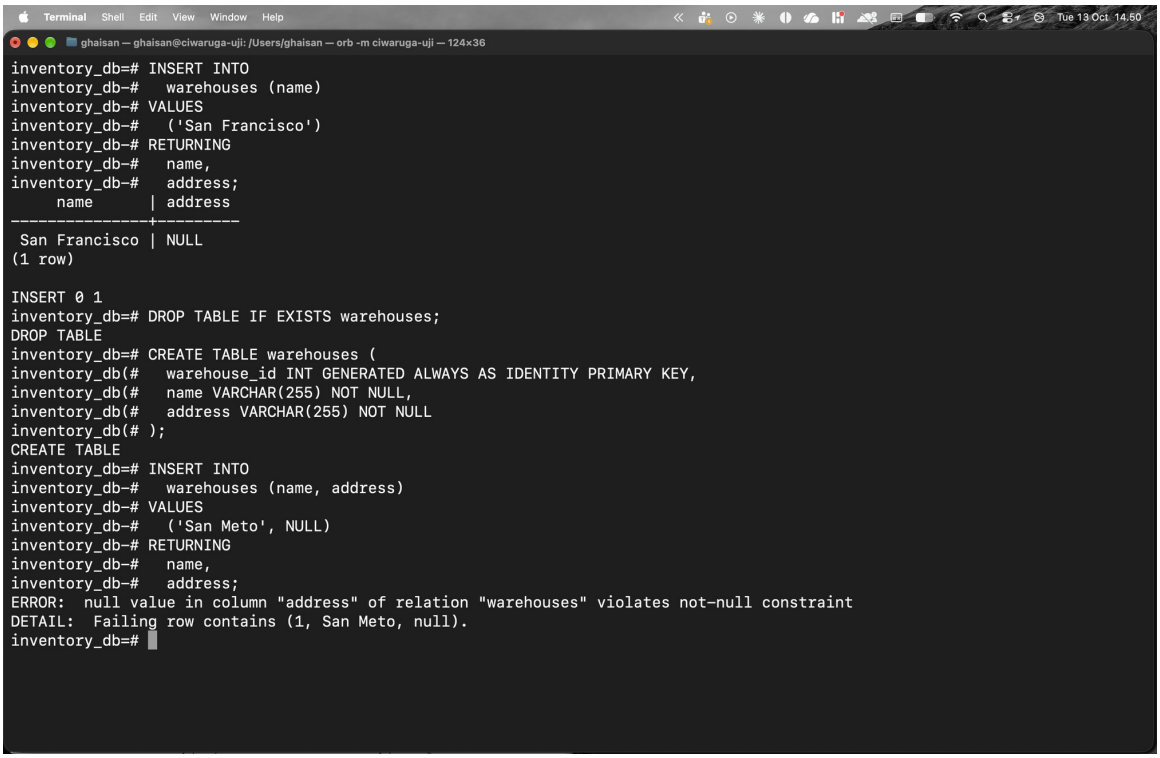
CREATE TABLE warehouses (
  warehouse_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  address VARCHAR(255) NOT NULL
);

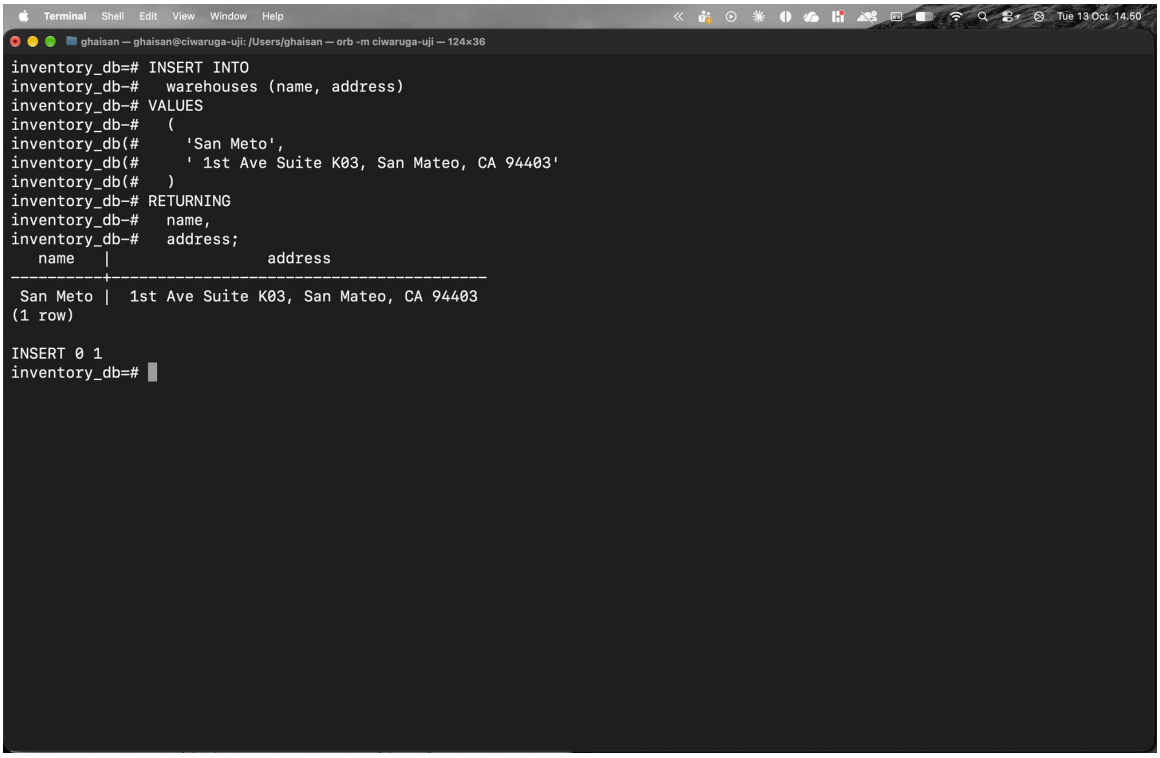
INSERT INTO
  warehouses (name, address)
VALUES
  ('San Meto', NULL)
RETURNING
  name,
  address;

INSERT INTO
  warehouses (name, address)
VALUES
  (
    'San Meto',
    ' 1st Ave Suite K03, San Mateo, CA 94403'
  )
RETURNING
  name,
  address;
```

Keterangan
NULL = 1 dan NULL = NULL sama-sama menghasilkan NULL, karena membandingkan nilai yang tidak diketahui hasilnya juga tidak diketahui. Tabel warehouses pertama dibuat tanpa NOT NULL, jadi kolom address boleh diisi NULL (San Jose), dan kalau kolom tidak disebut di INSERT nilainya juga otomatis NULL (San Francisco). Setelah itu tabel warehouses di-drop lalu dibuat ulang dengan NOT NULL pada name dan address. Insert San Meto dengan address NULL ditolak dengan error null value in column "address" of relation "warehouses" violates not-null constraint. Insert yang alamatnya diisi lengkap berhasil.
Hasil Eksekusi (Screenshot)




Terminal Shell Edit View Window Help
Tue 13 Oct 14.50
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# INSERT INTO
inventory_db-#   warehouses (name)
inventory_db-# VALUES
inventory_db-#   ('San Francisco')
inventory_db-# RETURNING
inventory_db-#   name,
inventory_db-#   address;
name | address
San Francisco | NULL
(1 row)
INSERT 0 1
inventory_db=# DROP TABLE IF EXISTS warehouses;
DROP TABLE
inventory_db=# CREATE TABLE warehouses (
inventory_db(#   warehouse_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(255) NOT NULL,
inventory_db(#   address VARCHAR(255) NOT NULL
inventory_db(#);
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   warehouses (name, address)
inventory_db-# VALUES
inventory_db-#   ('San Meto', NULL)
inventory_db-# RETURNING
inventory_db-#   name,
inventory_db-#   address;
ERROR: null value in column "address" of relation "warehouses" violates not-null constraint
DETAIL: Failing row contains (1, San Meto, null).
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.50
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# INSERT INTO
inventory_db-#   warehouses (name, address)
inventory_db-# VALUES
inventory_db-#   (
inventory_db(#   'San Meto',
inventory_db(#   ' 1st Ave Suite K03, San Mateo, CA 94403'
inventory_db(#   )
inventory_db-# RETURNING
inventory_db-#   name,
inventory_db-#   address;
name | address
San Meto | 1st Ave Suite K03, San Mateo, CA 94403
(1 row)
INSERT 0 1
inventory_db=#

Latihan No. 10

Judul

Section 3: DEFAULT constraint

Syntax

```
CREATE TABLE items (
  item_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  quantity INT NOT NULL,
  price DEC(11, 2) NOT NULL,
  tax DEC(11, 2) DEFAULT 0.05
);

INSERT INTO
  items (name, quantity, price)
VALUES
  ('iPhone 15 Pro', 1, 1299.99) RETURNING *;

INSERT INTO
  items (name, quantity, price, tax)
VALUES
  ('iPhone 16 Pro', 1, 1399.99, DEFAULT)
RETURNING *;

INSERT INTO
  items (name, quantity, price, tax)
VALUES
  ('iPhone 17 Pro', 1, 1499.99, 0.08)
RETURNING *;

CREATE TABLE orders (
  order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer VARCHAR(100) NOT NULL,
  ship_to VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

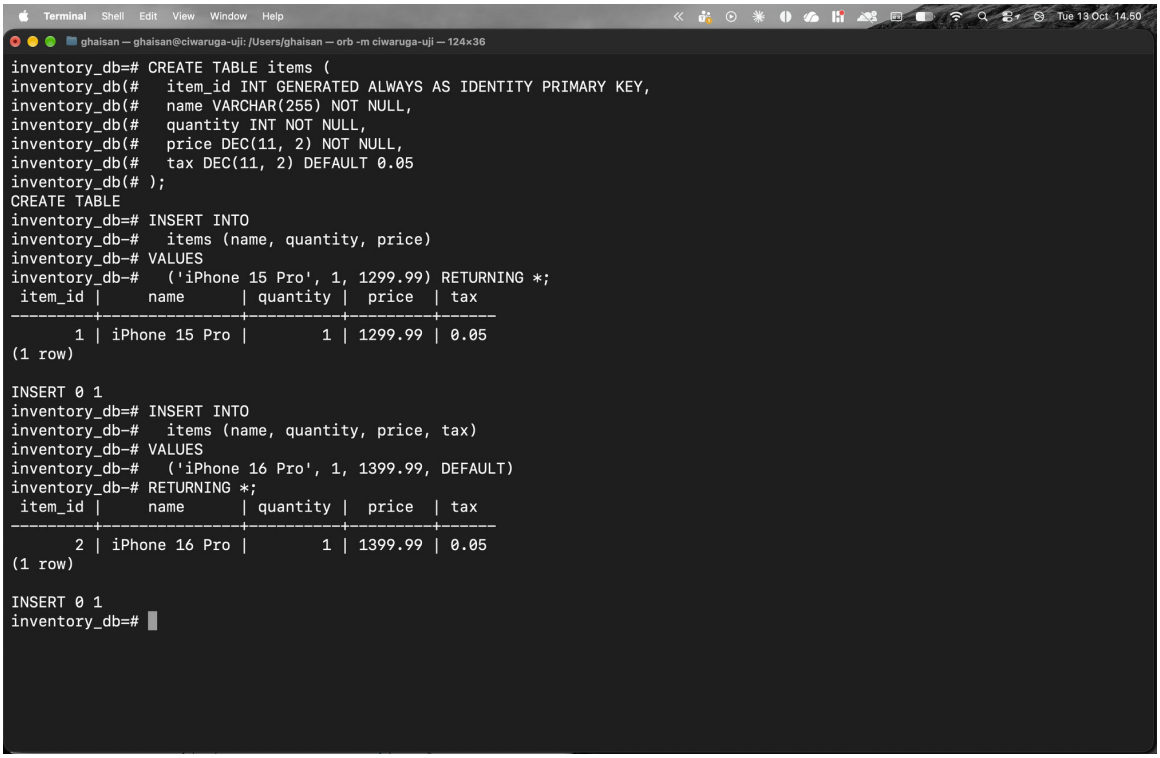
INSERT INTO
  orders (customer, ship_to)
VALUES
  (
    'John Doe',
    '9000 N 1st Street, San Jose, CA 95134'
  )
RETURNING *;
```

Keterangan

Kolom tax di tabel items punya DEFAULT 0.05. Insert iPhone 15 Pro tanpa menyebut kolom tax menghasilkan tax 0.05. Insert iPhone 16 Pro memakai kata kunci DEFAULT di VALUES, hasilnya juga 0.05. Insert iPhone 17 Pro mengisi tax 0.08 sendiri, jadi nilai default tidak dipakai.

Tabel orders punya kolom created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP. Saat pesanan John Doe dimasukkan tanpa created_at, kolom itu terisi waktu ketika query dijalankan (2026-10-13 14:50:21), sama dengan jam yang tampil di screenshot.

Hasil Eksekusi (Screenshot)

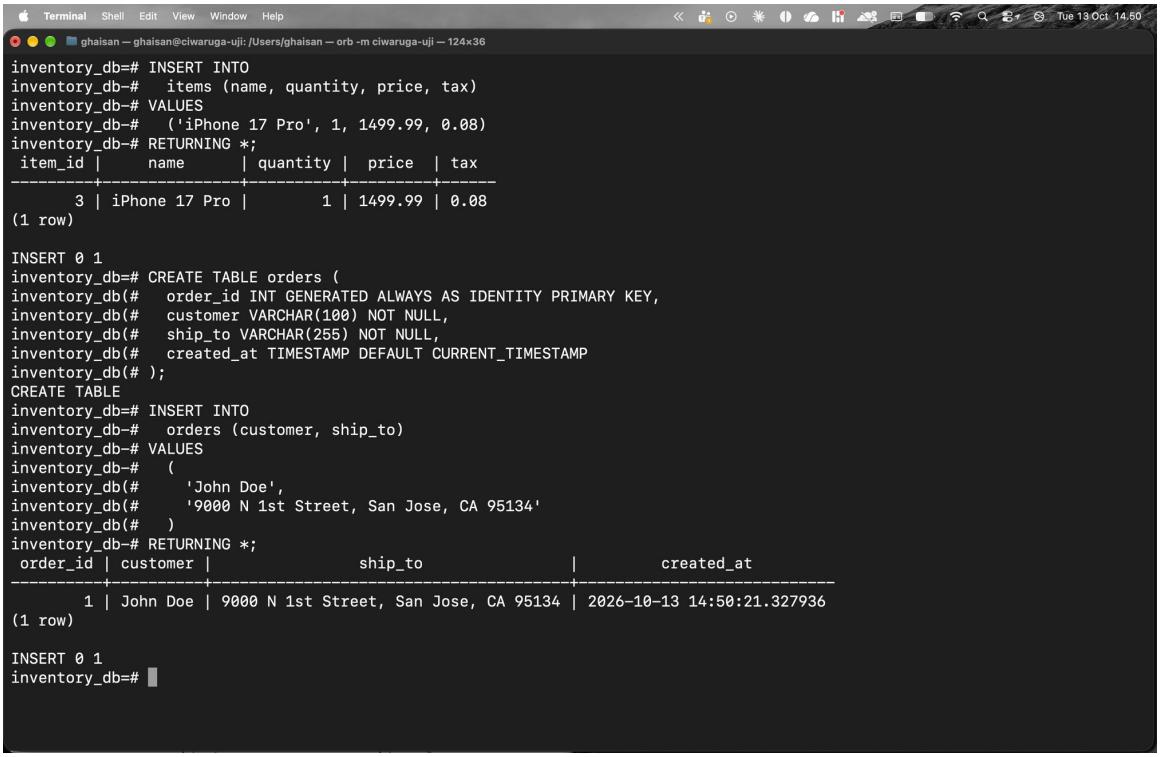


```
Terminal  Shell  Edit  View  Window  Help                                  Tue 13 Oct  14.50
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36

inventory_db=# CREATE TABLE items (
inventory_db(#   item_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(255) NOT NULL,
inventory_db(#   quantity INT NOT NULL,
inventory_db(#   price DEC(11, 2) NOT NULL,
inventory_db(#   tax DEC(11, 2) DEFAULT 0.05
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   items (name, quantity, price)
inventory_db-# VALUES
inventory_db-#   ('iPhone 15 Pro', 1, 1299.99) RETURNING *;
 item_id |     name      | quantity |  price  | tax
---------+---------------+----------+---------+------
       1 | iPhone 15 Pro |        1 | 1299.99 | 0.05
(1 row)

INSERT 0 1
inventory_db=# INSERT INTO
inventory_db-#   items (name, quantity, price, tax)
inventory_db-# VALUES
inventory_db-#   ('iPhone 16 Pro', 1, 1399.99, DEFAULT)
inventory_db-# RETURNING *;
 item_id |     name      | quantity |  price  | tax
---------+---------------+----------+---------+------
       2 | iPhone 16 Pro |        1 | 1399.99 | 0.05
(1 row)

INSERT 0 1
inventory_db=#
```



```
Terminal  Shell  Edit  View  Window  Help                                  Tue 13 Oct  14.50
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36

inventory_db=# INSERT INTO
inventory_db-#   items (name, quantity, price, tax)
inventory_db-# VALUES
inventory_db-#   ('iPhone 17 Pro', 1, 1499.99, 0.08)
inventory_db-# RETURNING *;
 item_id |     name      | quantity |  price  | tax
---------+---------------+----------+---------+------
       3 | iPhone 17 Pro |        1 | 1499.99 | 0.08
(1 row)

INSERT 0 1
inventory_db=# CREATE TABLE orders (
inventory_db(#   order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   customer VARCHAR(100) NOT NULL,
inventory_db(#   ship_to VARCHAR(255) NOT NULL,
inventory_db(#   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   orders (customer, ship_to)
inventory_db-# VALUES
inventory_db-#   (
inventory_db(#     'John Doe',
inventory_db(#     '9000 N 1st Street, San Jose, CA 95134'
inventory_db(#   )
inventory_db-# RETURNING *;
 order_id | customer |               ship_to                |         created_at
----------+----------+--------------------------------------+----------------------------
        1 | John Doe | 9000 N 1st Street, San Jose, CA 95134 | 2026-10-13 14:50:21.327936
(1 row)

INSERT 0 1
inventory_db=#
```

Latihan No. 11

Judul

Section 3: CHECK constraint

Syntax

```
CREATE TABLE products (
  product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  price DEC(11, 2) NOT NULL CONSTRAINT positive_price CHECK (price >= 0),
  discounted_price DEC(11, 2) NOT NULL DEFAULT 0
);

INSERT INTO
  products (name, price)
VALUES
  ('iPhone Pro 15', -1299.99);

INSERT INTO
  products (name, price)
VALUES
  ('iPhone Pro 15', 1299.99) RETURNING *;

UPDATE products
SET
  price = -899.99
WHERE
  product_id = 2;

UPDATE products
SET
  price = 999.99
WHERE
  product_id = 2
RETURNING *;

ALTER TABLE products
ADD CONSTRAINT discounted_price_check
CHECK (discounted_price < price AND discounted_price > 0);

UPDATE products
SET discounted_price = price * 0.9
RETURNING *;

ALTER TABLE products
ADD CONSTRAINT discounted_price_check
CHECK (discounted_price < price AND discounted_price > 0);

INSERT INTO products(name, price, discounted_price)
VALUES('iPhone Pro 15', 1299.99, 1399.99);

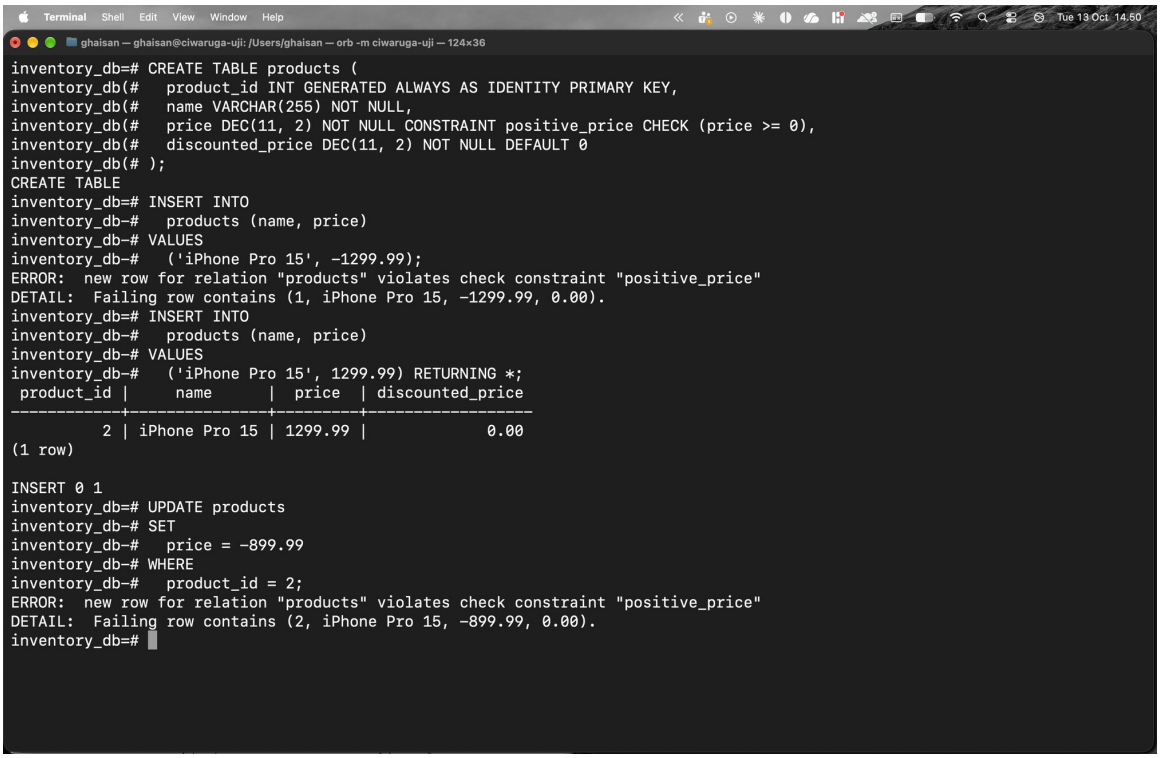
ALTER TABLE products
DROP CONSTRAINT IF EXISTS discounted_price_check;
```

Keterangan

Tabel products punya CHECK bernama positive_price (price >= 0). Insert harga -1299.99 ditolak karena melanggar positive_price. Insert harga 1299.99 berhasil dan mendapat product_id 2, karena nilai 1 dari identity sudah terpakai oleh insert yang gagal. Update harga menjadi -899.99 juga ditolak, sedangkan update menjadi 999.99 berhasil.

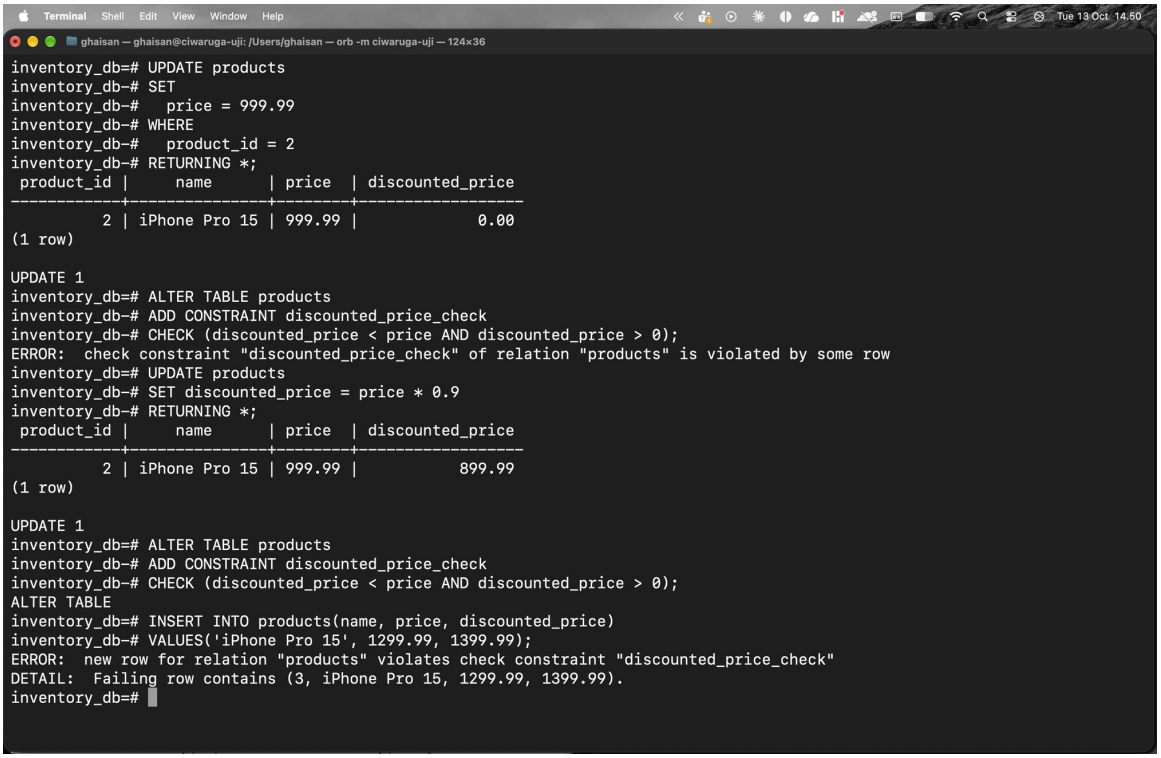
ALTER TABLE ... ADD CONSTRAINT discounted_price_check (discounted_price < price AND discounted_price > 0) pertama kali ditolak karena baris yang sudah ada punya discounted_price 0. Setelah discounted_price diupdate menjadi price * 0.9 (899.99), constraint berhasil ditambahkan. Insert dengan discounted_price 1399.99 yang lebih besar dari price kemudian ditolak. Terakhir constraint dihapus dengan ALTER TABLE ... DROP CONSTRAINT IF EXISTS.

Hasil Eksekusi (Screenshot)



```
inventory_db=# CREATE TABLE products (
inventory_db(#   product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(255) NOT NULL,
inventory_db(#   price DEC(11, 2) NOT NULL CONSTRAINT positive_price CHECK (price >= 0),
inventory_db(#   discounted_price DEC(11, 2) NOT NULL DEFAULT 0
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   products (name, price)
inventory_db-# VALUES
inventory_db-#   ('iPhone Pro 15', -1299.99);
ERROR:  new row for relation "products" violates check constraint "positive_price"
DETAIL:  Failing row contains (1, iPhone Pro 15, -1299.99, 0.00).
inventory_db=# INSERT INTO
inventory_db-#   products (name, price)
inventory_db-# VALUES
inventory_db-#   ('iPhone Pro 15', 1299.99) RETURNING *;
 product_id |     name      |  price  | discounted_price
------------+---------------+---------+------------------
          2 | iPhone Pro 15 | 1299.99 |             0.00
(1 row)

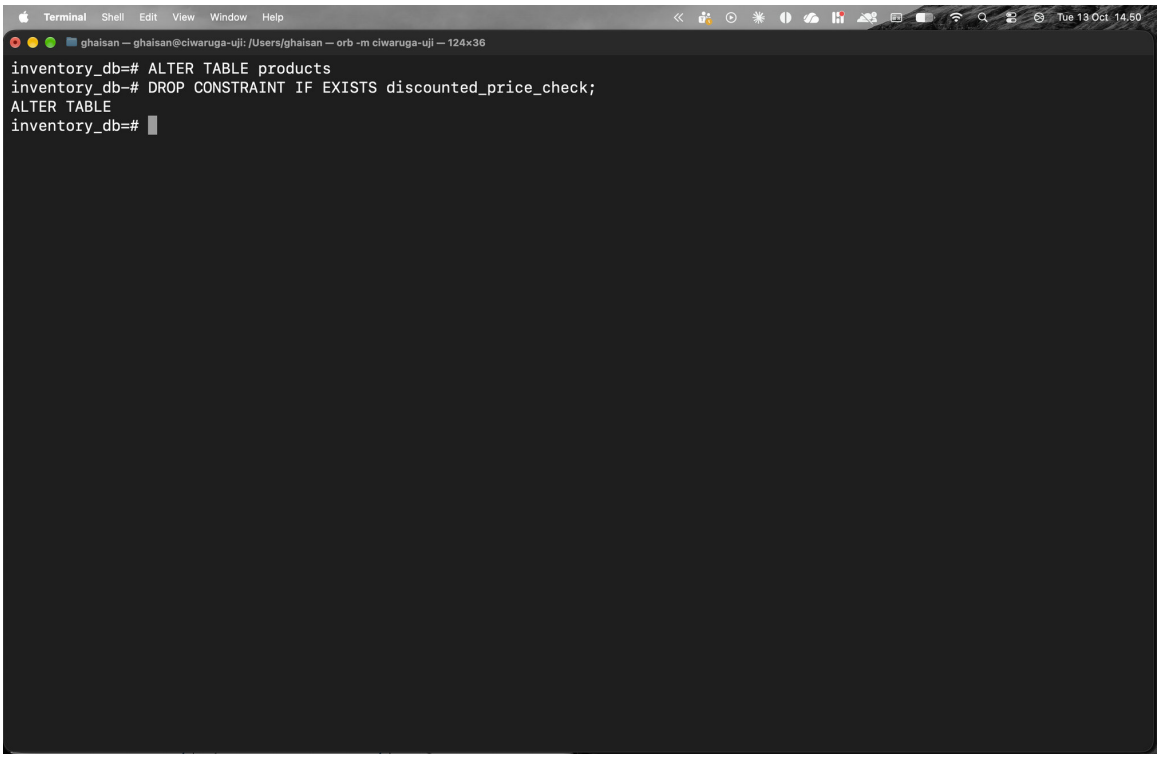
INSERT 0 1
inventory_db=# UPDATE products
inventory_db-# SET
inventory_db-#   price = -899.99
inventory_db-# WHERE
inventory_db-#   product_id = 2;
ERROR:  new row for relation "products" violates check constraint "positive_price"
DETAIL:  Failing row contains (2, iPhone Pro 15, -899.99, 0.00).
inventory_db=#
```



```
inventory_db=# UPDATE products
inventory_db-# SET
inventory_db-#   price = 999.99
inventory_db-# WHERE
inventory_db-#   product_id = 2
inventory_db-# RETURNING *;
 product_id |     name      | price  | discounted_price
------------+---------------+--------+------------------
          2 | iPhone Pro 15 | 999.99 |             0.00
(1 row)

UPDATE 1
inventory_db=# ALTER TABLE products
inventory_db-# ADD CONSTRAINT discounted_price_check
inventory_db-# CHECK (discounted_price < price AND discounted_price > 0);
ERROR:  check constraint "discounted_price_check" of relation "products" is violated by some row
inventory_db=# UPDATE products
inventory_db-# SET discounted_price = price * 0.9
inventory_db-# RETURNING *;
 product_id |     name      | price  | discounted_price
------------+---------------+--------+------------------
          2 | iPhone Pro 15 | 999.99 |           899.99
(1 row)

UPDATE 1
inventory_db=# ALTER TABLE products
inventory_db-# ADD CONSTRAINT discounted_price_check
inventory_db-# CHECK (discounted_price < price AND discounted_price > 0);
ALTER TABLE
inventory_db=# INSERT INTO products(name, price, discounted_price)
inventory_db-# VALUES('iPhone Pro 15', 1299.99, 1399.99);
ERROR:  new row for relation "products" violates check constraint "discounted_price_check"
DETAIL:  Failing row contains (3, iPhone Pro 15, 1299.99, 1399.99).
inventory_db=#
```



Latihan No. 12

Judul
Section 3: UNIQUE constraint
Syntax

```
DROP TABLE IF EXISTS brands;

CREATE TABLE brands (
  brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(255) NOT NULL UNIQUE
);

INSERT INTO
  brands (name)
VALUES
  ('Apple')
RETURNING *;

INSERT INTO
  brands (name)
VALUES
  ('Apple')
RETURNING *;

CREATE TABLE product_loans (
  loan_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id INT NOT NULL,
  user_id INT NOT NULL,
  loan_date DATE NOT NULL,
  return_date DATE,
  UNIQUE (product_id, user_id)
);

INSERT INTO
```

```
  product_loans (product_id, user_id, loan_date)
VALUES
  (1, 1, '2024-11-23'),
  (1, 2, '2024-11-23')
RETURNING *;

INSERT INTO
  product_loans (product_id, user_id, loan_date)
VALUES
  (1, 1, '2024-11-24');

CREATE TABLE profiles (
  user_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone VARCHAR(25) UNIQUE
);

INSERT INTO
  profiles (user_id, first_name, last_name, phone)
VALUES
  (1, 'John', 'Doe', NULL)
RETURNING *;

INSERT INTO
  profiles (user_id, first_name, last_name, phone)
VALUES
  (2, 'Jane', 'Doe', NULL)
RETURNING *;

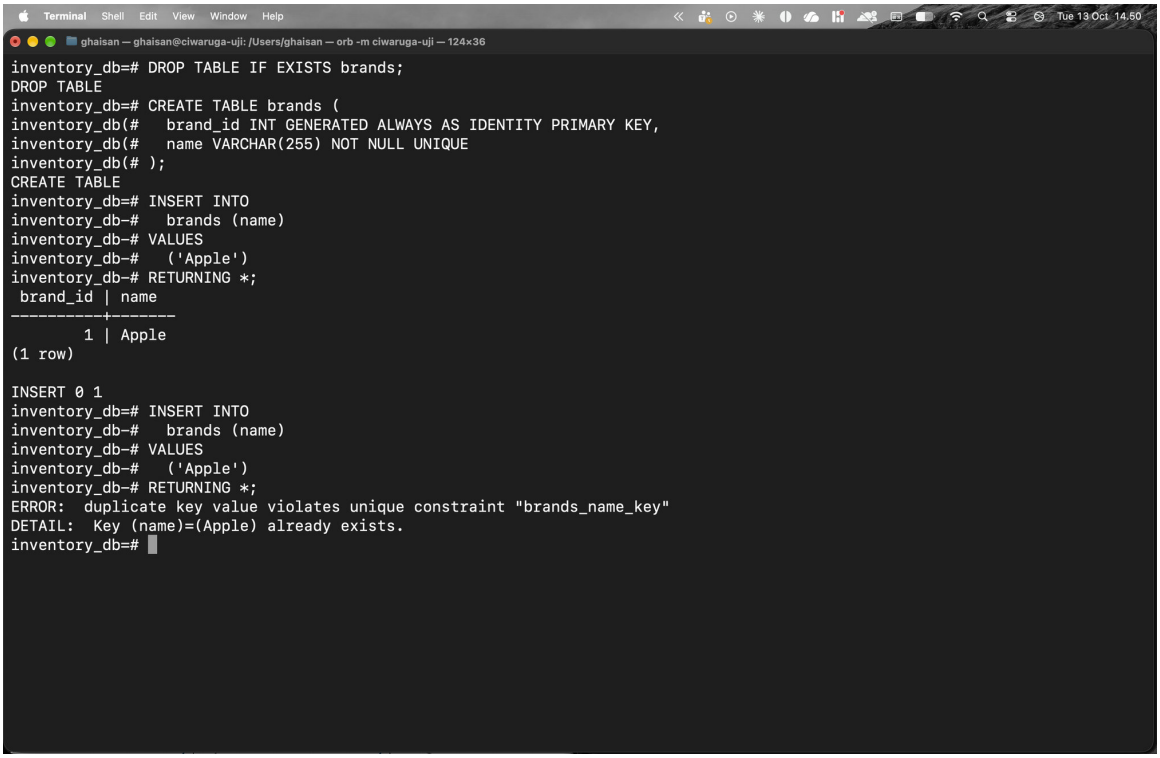
SELECT * FROM profiles;
```

Keterangan

Tabel brands dari latihan primary key dihapus dulu (DROP TABLE IF EXISTS brands) karena pada materi ini brands dibuat ulang dengan kolom name yang UNIQUE. Insert Apple pertama berhasil, sedangkan insert Apple kedua ditolak dengan error duplicate key value violates unique constraint "brands_name_key". Nama constraint dibuat otomatis dengan format tabel_kolom_key.

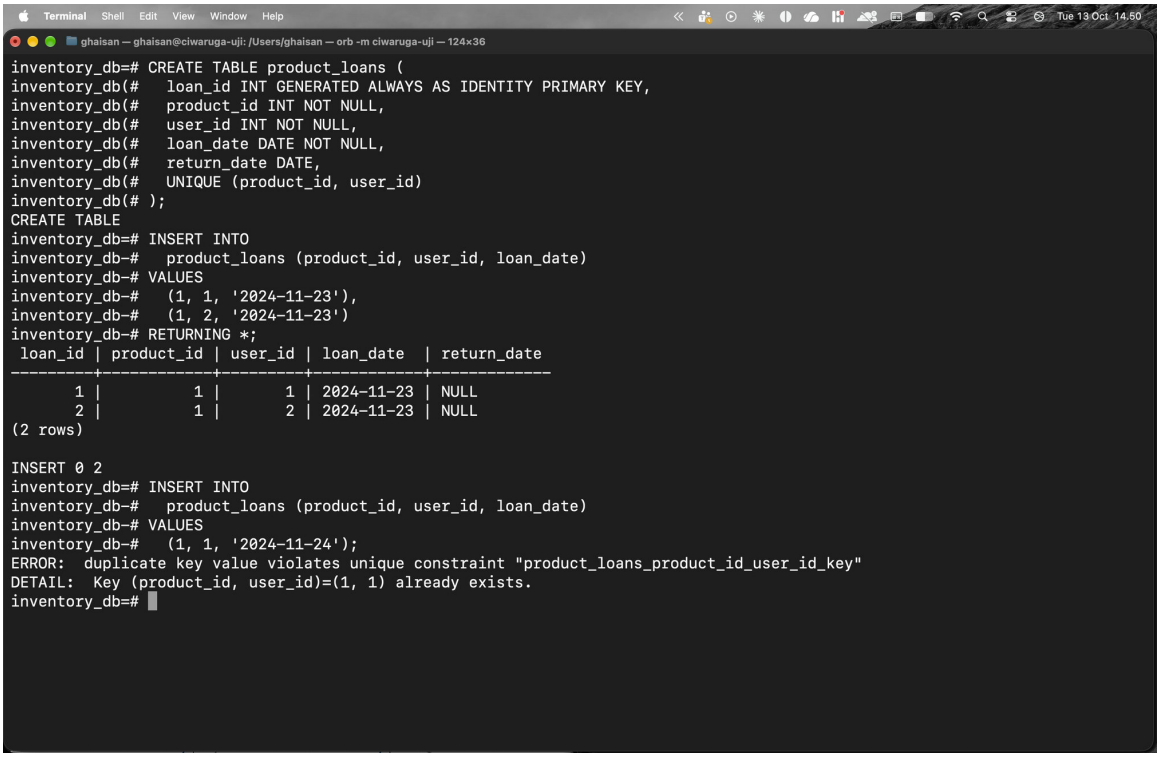
Tabel product_loans memakai UNIQUE (product_id, user_id) untuk dua kolom. Pasangan (1, 1) dan (1, 2) boleh masuk, tetapi (1, 1) yang kedua ditolak oleh constraint product_loans_product_id_user_id_key. Pada tabel profiles kolom phone UNIQUE tetapi boleh NULL. Dua baris dengan phone NULL (John dan Jane) sama-sama berhasil masuk karena di PostgreSQL NULL tidak dianggap sama dengan NULL lain, terlihat dari SELECT * FROM profiles.

Hasil Eksekusi (Screenshot)



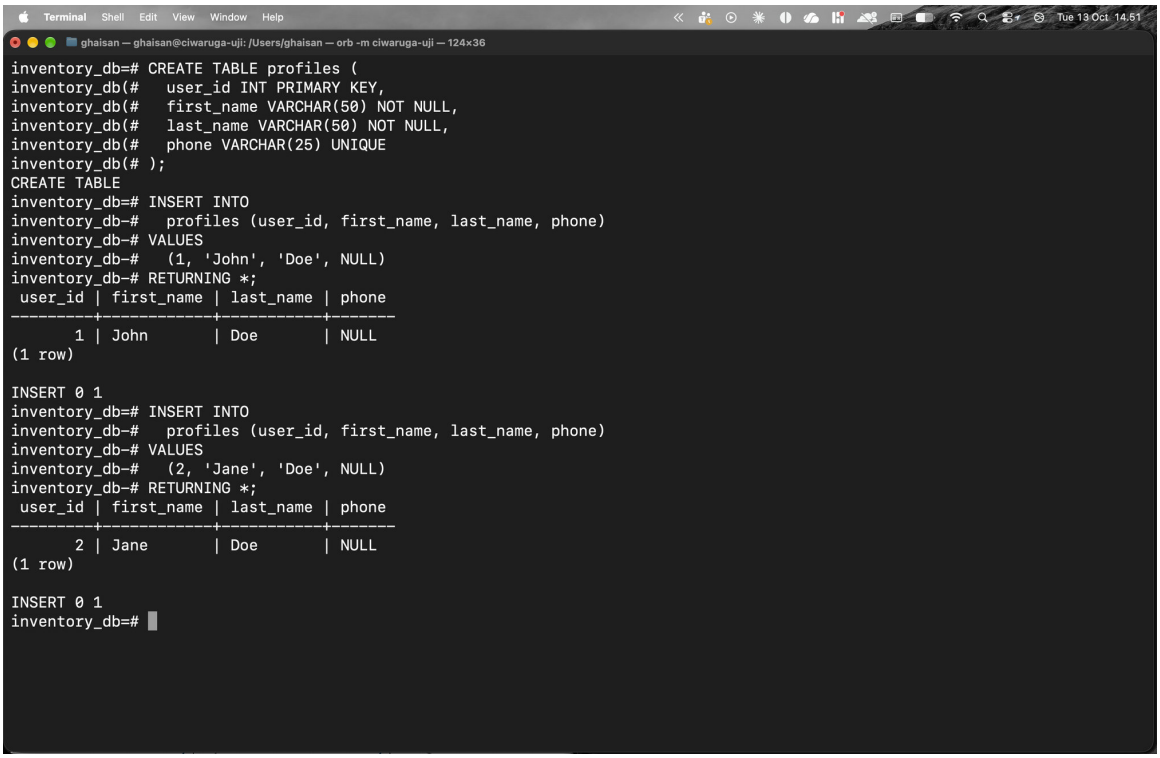
```
inventory_db=# DROP TABLE IF EXISTS brands;
DROP TABLE
inventory_db=# CREATE TABLE brands (
inventory_db(#   brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(255) NOT NULL UNIQUE
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   brands (name)
inventory_db-# VALUES
inventory_db-#   ('Apple')
inventory_db-# RETURNING *;
 brand_id | name
----------+-------
        1 | Apple
(1 row)

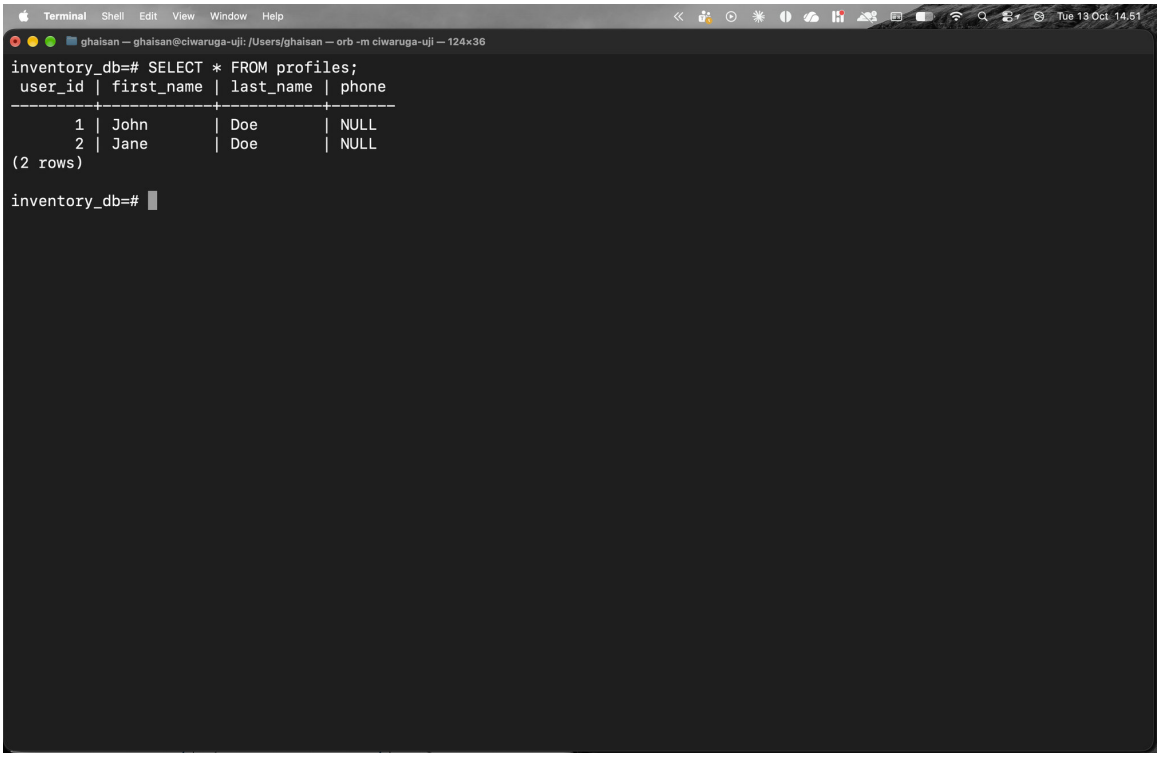
INSERT 0 1
inventory_db=# INSERT INTO
inventory_db-#   brands (name)
inventory_db-# VALUES
inventory_db-#   ('Apple')
inventory_db-# RETURNING *;
ERROR:  duplicate key value violates unique constraint "brands_name_key"
DETAIL:  Key (name)=(Apple) already exists.
inventory_db=#
```



```
inventory_db=# CREATE TABLE product_loans (
inventory_db(#   loan_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   product_id INT NOT NULL,
inventory_db(#   user_id INT NOT NULL,
inventory_db(#   loan_date DATE NOT NULL,
inventory_db(#   return_date DATE,
inventory_db(#   UNIQUE (product_id, user_id)
inventory_db(# );
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   product_loans (product_id, user_id, loan_date)
inventory_db-# VALUES
inventory_db-#   (1, 1, '2024-11-23'),
inventory_db-#   (1, 2, '2024-11-23')
inventory_db-# RETURNING *;
 loan_id | product_id | user_id | loan_date  | return_date
---------+------------+---------+------------+-------------
       1 |          1 |       1 | 2024-11-23 | NULL
       2 |          1 |       2 | 2024-11-23 | NULL
(2 rows)

INSERT 0 2
inventory_db=# INSERT INTO
inventory_db-#   product_loans (product_id, user_id, loan_date)
inventory_db-# VALUES
inventory_db-#   (1, 1, '2024-11-24');
ERROR:  duplicate key value violates unique constraint "product_loans_product_id_user_id_key"
DETAIL:  Key (product_id, user_id)=(1, 1) already exists.
inventory_db=#
```


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# CREATE TABLE profiles (
inventory_db(#   user_id INT PRIMARY KEY,
inventory_db(#   first_name VARCHAR(50) NOT NULL,
inventory_db(#   last_name VARCHAR(50) NOT NULL,
inventory_db(#   phone VARCHAR(25) UNIQUE
inventory_db(#);
CREATE TABLE
inventory_db=# INSERT INTO
inventory_db-#   profiles (user_id, first_name, last_name, phone)
inventory_db-# VALUES
inventory_db-#   (1, 'John', 'Doe', NULL)
inventory_db-# RETURNING *;
 user_id | first_name | last_name | phone
---------+------------+-----------+-------
       1 | John | Doe | NULL
(1 row)
INSERT 0 1
inventory_db=# INSERT INTO
inventory_db-#   profiles (user_id, first_name, last_name, phone)
inventory_db-# VALUES
inventory_db-#   (2, 'Jane', 'Doe', NULL)
inventory_db-# RETURNING *;
 user_id | first_name | last_name | phone
---------+------------+-----------+-------
       2 | Jane | Doe | NULL
(1 row)
INSERT 0 1
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# SELECT * FROM profiles;
 user_id | first_name | last_name | phone
---------+------------+-----------+-------
       1 | John | Doe | NULL
       2 | Jane | Doe | NULL
(2 rows)
inventory_db=#

Latihan No. 13

Judul
Section 3: Foreign key

Syntax

```
DROP TABLE IF EXISTS products, brands;

CREATE TABLE brands (
    brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE products (
   product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   price DECIMAL(10, 2) NOT NULL,
   brand_id INT NOT NULL,
   FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
);

INSERT INTO brands(name)
VALUES ('Apple'), ('Samsung'), ('Google')
RETURNING *;

INSERT INTO products(name, price, brand_id)
VALUES('iPhone 14 Pro', 999.99, 1)
RETURNING *;

INSERT INTO products(name, price, brand_id)
VALUES('iPhone 15 Pro', 1299.99, 11)
RETURNING *;

DROP TABLE brands;

DROP TABLE brands CASCADE;

DROP TABLE products;

CREATE TABLE brands (
    brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE products (
  product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  brand_id INT,
  FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
  ON DELETE SET NULL
);

INSERT INTO brands(name)
VALUES ('Apple'), ('Samsung')
RETURNING *;

INSERT INTO products(name, price, brand_id)
VALUES
('iPhone 14 Pro', 999.99, 1),
('iPhone 15 Pro', 1299.99, 1),
('Galaxy S23 Ultra', 1299.99, 2)
RETURNING *;

DELETE FROM brands WHERE brand_id = 1;
```

```
SELECT * FROM products;

DROP TABLE brands, products;

CREATE TABLE brands (
    brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE products (
  product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  brand_id INT,
  FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
  ON DELETE CASCADE
);

INSERT INTO brands(name)
VALUES('Apple'), ('Samsung')
RETURNING *;

INSERT INTO products(name, price, brand_id)
VALUES
('iPhone 14 Pro', 999.99, 1),
('iPhone 15 Pro', 1299.99, 1),
('Galaxy S23 Ultra', 1299.99, 2)
RETURNING *;

DELETE FROM brands
WHERE brand_id = 1;

SELECT * FROM products;

DROP TABLE brands, products;

CREATE TABLE brands (
    brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(255) NOT NULL
);

CREATE TABLE products (
  product_id INT GENERATED ALWAYS AS IDENTITY  PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  brand_id INT DEFAULT 1,
  FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
  ON DELETE SET DEFAULT
);

INSERT INTO brands(name)
VALUES('Unknown'),
      ('Apple'),
      ('Samsung')
RETURNING *;

INSERT INTO products(name, price, brand_id)
VALUES
('iPhone 14 Pro', 999.99, 2),
('iPhone 15 Pro', 1299.99, 2),
('Galaxy S23 Ultra', 1299.99, 3)
RETURNING *;

DELETE FROM brands
WHERE brand_id = 2;
```

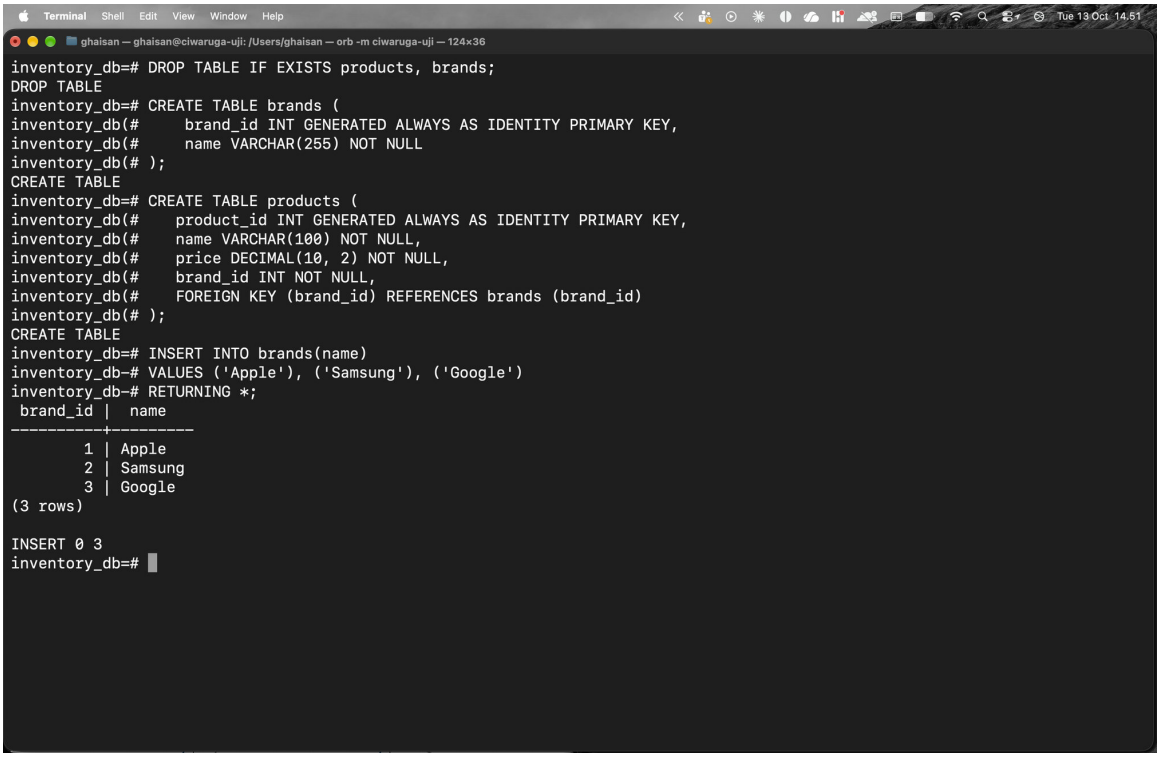
```
SELECT * FROM products;
```

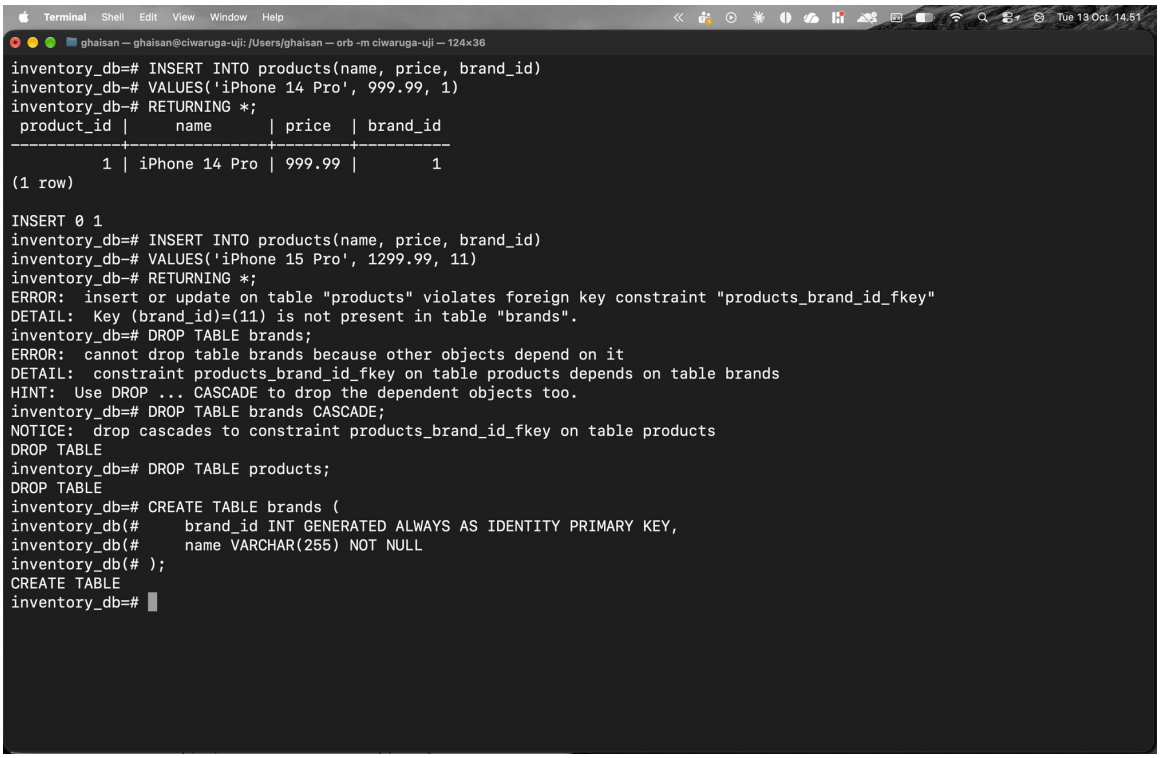
Keterangan

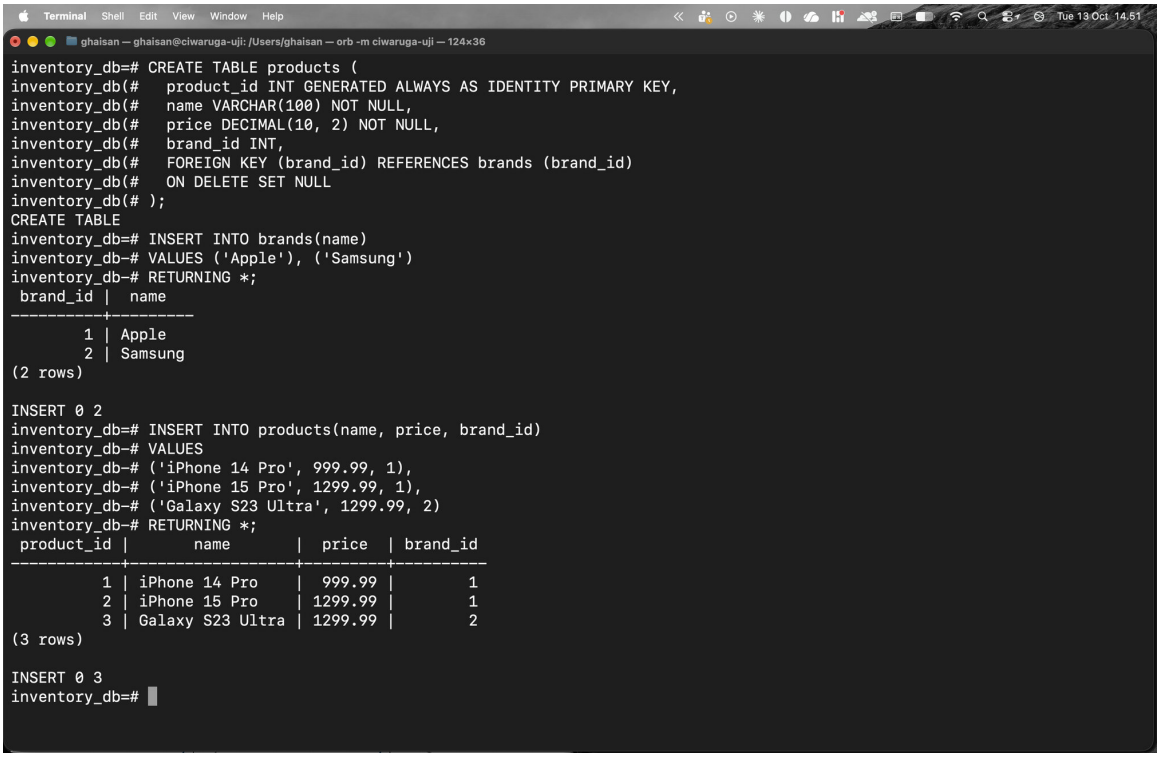
Tabel products (dari latihan CHECK) dan brands (dari latihan UNIQUE) dihapus dulu supaya bisa dibuat ulang sesuai materi. Tabel products punya FOREIGN KEY (brand_id) REFERENCES brands (brand_id). Insert produk dengan brand_id 1 berhasil karena brand 1 (Apple) ada, sedangkan brand_id 11 ditolak oleh constraint products_brand_id_fkey karena tidak ada di tabel brands. DROP TABLE brands gagal karena masih direferensikan oleh products, sedangkan DROP TABLE brands CASCADE berhasil sambil menghapus constraint foreign key di products.

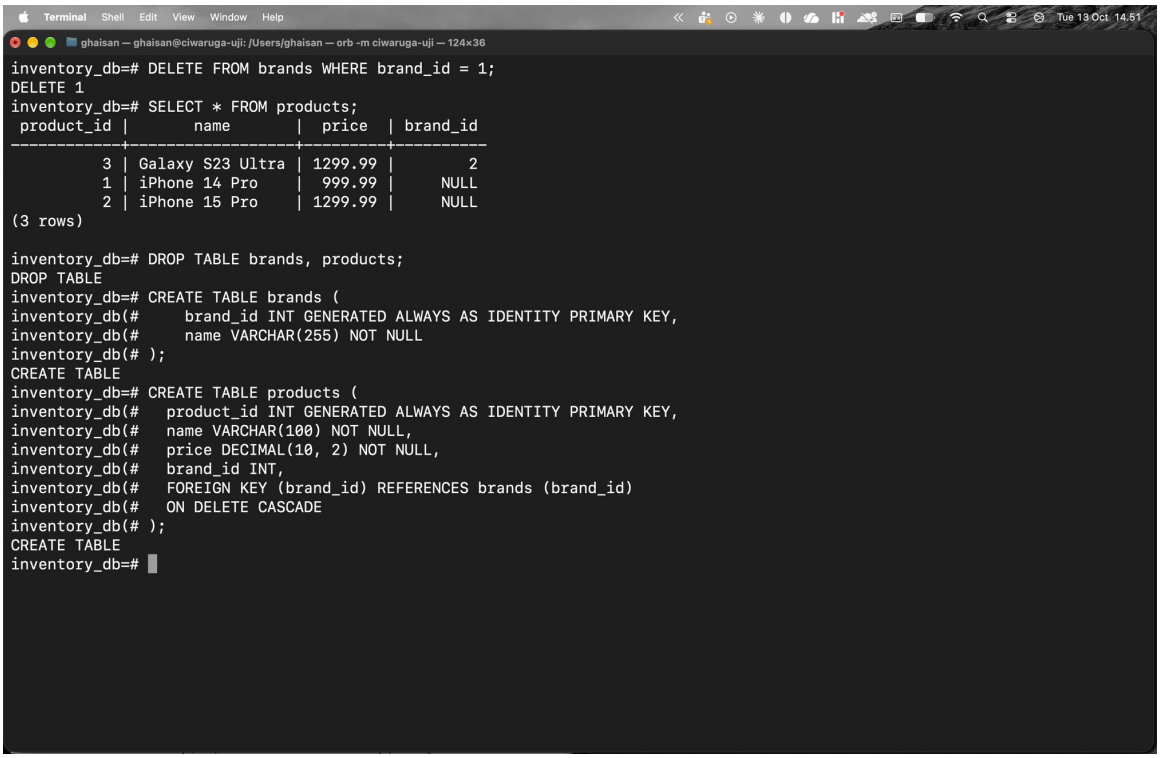
Berikutnya dicoba tiga aksi ON DELETE dengan data yang sama (brand Apple dan Samsung, tiga produk). Pada ON DELETE SET NULL, setelah brand 1 dihapus, brand_id iPhone 14 Pro dan iPhone 15 Pro berubah menjadi NULL. Pada ON DELETE CASCADE, kedua produk Apple ikut terhapus sehingga tinggal Galaxy S23 Ultra. Pada ON DELETE SET DEFAULT, brand_id punya DEFAULT 1 (brand Unknown), sehingga setelah brand 2 (Apple) dihapus brand_id kedua iPhone berubah menjadi 1.

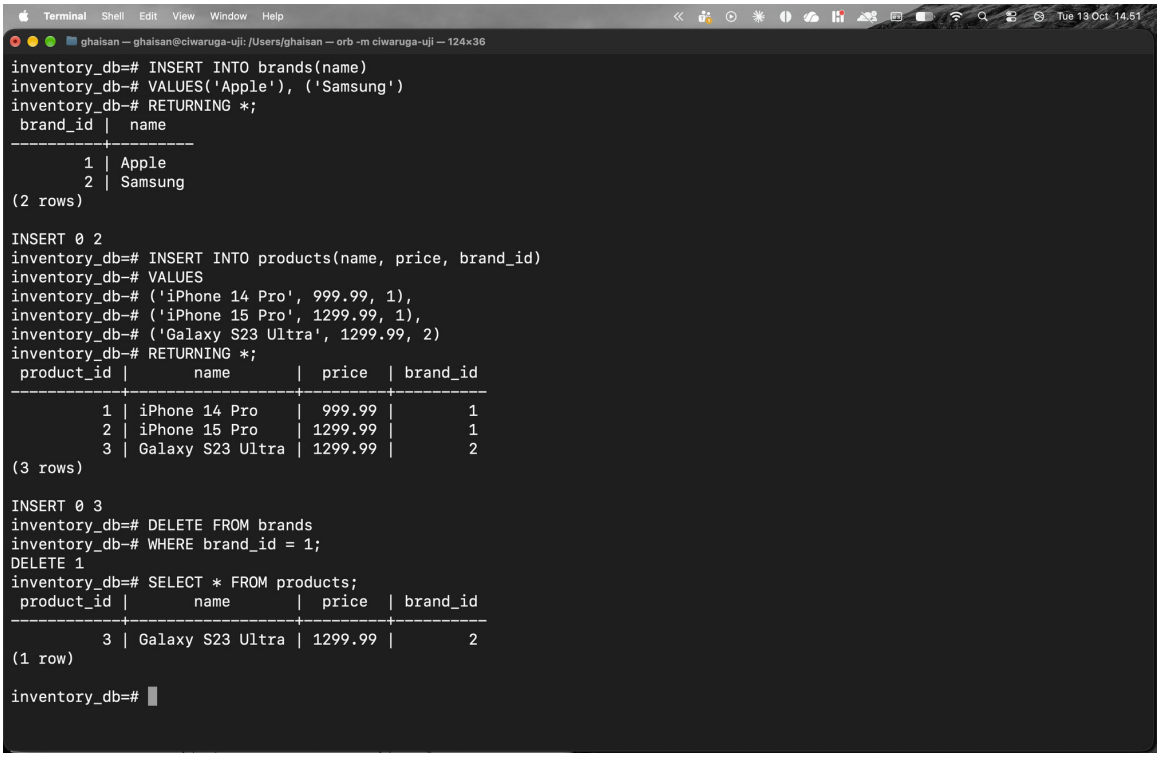
Hasil Eksekusi (Screenshot)

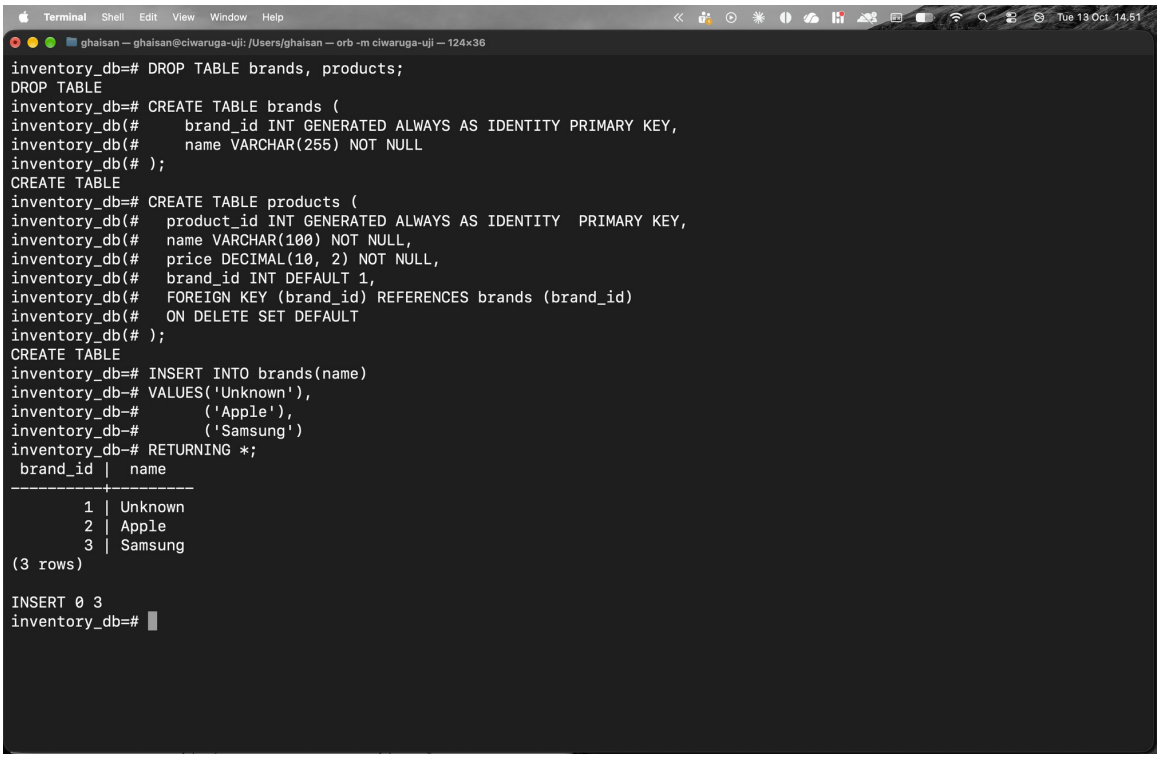


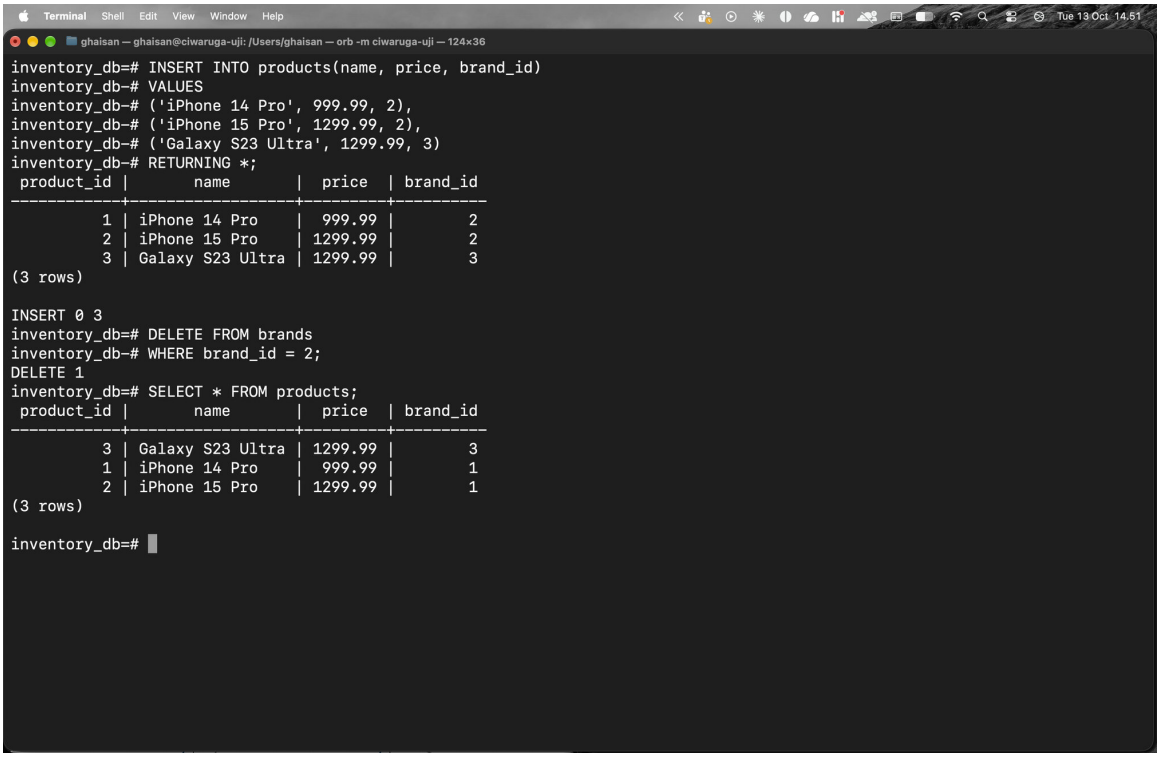

Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# INSERT INTO products(name, price, brand_id)
inventory_db-# VALUES('iPhone 14 Pro', 999.99, 1)
inventory_db-# RETURNING *;
product_id | name | price | brand_id
1 | iPhone 14 Pro | 999.99 | 1
(1 row)
INSERT 0 1
inventory_db=# INSERT INTO products(name, price, brand_id)
inventory_db-# VALUES('iPhone 15 Pro', 1299.99, 11)
inventory_db-# RETURNING *;
ERROR: insert or update on table "products" violates foreign key constraint "products_brand_id_fkey"
DETAIL: Key (brand_id)=(11) is not present in table "brands".
inventory_db=# DROP TABLE brands;
ERROR: cannot drop table brands because other objects depend on it
DETAIL: constraint products_brand_id_fkey on table products depends on table brands
HINT: Use DROP ... CASCADE to drop the dependent objects too.
inventory_db=# DROP TABLE brands CASCADE;
NOTICE: drop cascades to constraint products_brand_id_fkey on table products
DROP TABLE
inventory_db=# DROP TABLE products;
DROP TABLE
inventory_db=# CREATE TABLE brands (
inventory_db(# brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(# name VARCHAR(255) NOT NULL
inventory_db(#);
CREATE TABLE
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# CREATE TABLE products (
inventory_db(# product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(# name VARCHAR(100) NOT NULL,
inventory_db(# price DECIMAL(10, 2) NOT NULL,
inventory_db(# brand_id INT,
inventory_db(# FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
inventory_db(# ON DELETE SET NULL
inventory_db(#);
CREATE TABLE
inventory_db=# INSERT INTO brands(name)
inventory_db-# VALUES ('Apple'), ('Samsung')
inventory_db-# RETURNING *;
brand_id | name
1 | Apple
2 | Samsung
(2 rows)
INSERT 0 2
inventory_db=# INSERT INTO products(name, price, brand_id)
inventory_db-# VALUES
inventory_db-# ('iPhone 14 Pro', 999.99, 1),
inventory_db-# ('iPhone 15 Pro', 1299.99, 1),
inventory_db-# ('Galaxy S23 Ultra', 1299.99, 2)
inventory_db-# RETURNING *;
product_id | name | price | brand_id
1 | iPhone 14 Pro | 999.99 | 1
2 | iPhone 15 Pro | 1299.99 | 1
3 | Galaxy S23 Ultra | 1299.99 | 2
(3 rows)
INSERT 0 3
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# DELETE FROM brands WHERE brand_id = 1;
DELETE 1
inventory_db=# SELECT * FROM products;
product_id | name | price | brand_id
3 | Galaxy S23 Ultra | 1299.99 | 2
1 | iPhone 14 Pro | 999.99 | NULL
2 | iPhone 15 Pro | 1299.99 | NULL
(3 rows)
inventory_db=# DROP TABLE brands, products;
DROP TABLE
inventory_db=# CREATE TABLE brands (
inventory_db(# brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(# name VARCHAR(255) NOT NULL
inventory_db(#);
CREATE TABLE
inventory_db=# CREATE TABLE products (
inventory_db(# product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(# name VARCHAR(100) NOT NULL,
inventory_db(# price DECIMAL(10, 2) NOT NULL,
inventory_db(# brand_id INT,
inventory_db(# FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
inventory_db(# ON DELETE CASCADE
inventory_db(#);
CREATE TABLE
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# INSERT INTO brands(name)
inventory_db-# VALUES('Apple'), ('Samsung')
inventory_db-# RETURNING *;
brand_id | name
1 | Apple
2 | Samsung
(2 rows)
INSERT 0 2
inventory_db=# INSERT INTO products(name, price, brand_id)
inventory_db-# VALUES
inventory_db-# ('iPhone 14 Pro', 999.99, 1),
inventory_db-# ('iPhone 15 Pro', 1299.99, 1),
inventory_db-# ('Galaxy S23 Ultra', 1299.99, 2)
inventory_db-# RETURNING *;
product_id | name | price | brand_id
1 | iPhone 14 Pro | 999.99 | 1
2 | iPhone 15 Pro | 1299.99 | 1
3 | Galaxy S23 Ultra | 1299.99 | 2
(3 rows)
INSERT 0 3
inventory_db=# DELETE FROM brands
inventory_db-# WHERE brand_id = 1;
DELETE 1
inventory_db=# SELECT * FROM products;
product_id | name | price | brand_id
3 | Galaxy S23 Ultra | 1299.99 | 2
(1 row)
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# DROP TABLE brands, products;
DROP TABLE
inventory_db=# CREATE TABLE brands (
inventory_db(#   brand_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(255) NOT NULL
inventory_db(#);
CREATE TABLE
inventory_db=# CREATE TABLE products (
inventory_db(#   product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
inventory_db(#   name VARCHAR(100) NOT NULL,
inventory_db(#   price DECIMAL(10, 2) NOT NULL,
inventory_db(#   brand_id INT DEFAULT 1,
inventory_db(#   FOREIGN KEY (brand_id) REFERENCES brands (brand_id)
inventory_db(#   ON DELETE SET DEFAULT
inventory_db(#);
CREATE TABLE
inventory_db=# INSERT INTO brands(name)
inventory_db-# VALUES('Unknown'),
inventory_db-#   ('Apple'),
inventory_db-#   ('Samsung')
inventory_db-# RETURNING *;
 brand_id | name
----------+---------
        1 | Unknown
        2 | Apple
        3 | Samsung
(3 rows)
INSERT 0 3
inventory_db=#


Terminal Shell Edit View Window Help
Tue 13 Oct 14.51
ghaisan — ghaisan@ciwaruga-uji: /Users/ghaisan — orb -m ciwaruga-uji — 124×36
inventory_db=# INSERT INTO products(name, price, brand_id)
inventory_db-# VALUES
inventory_db-# ('iPhone 14 Pro', 999.99, 2),
inventory_db-# ('iPhone 15 Pro', 1299.99, 2),
inventory_db-# ('Galaxy S23 Ultra', 1299.99, 3)
inventory_db-# RETURNING *;
 product_id | name | price | brand_id
------------+------------------+---------+----------
          1 | iPhone 14 Pro | 999.99 | 2
          2 | iPhone 15 Pro | 1299.99 | 2
          3 | Galaxy S23 Ultra | 1299.99 | 3
(3 rows)
INSERT 0 3
inventory_db=# DELETE FROM brands
inventory_db-# WHERE brand_id = 2;
DELETE 1
inventory_db=# SELECT * FROM products;
 product_id | name | price | brand_id
------------+------------------+---------+----------
          3 | Galaxy S23 Ultra | 1299.99 | 3
          1 | iPhone 14 Pro | 999.99 | 1
          2 | iPhone 15 Pro | 1299.99 | 1
(3 rows)
inventory_db=#